ESIEE Paris 2025-2026

Rapport de projet Truitteur

Les Asystes

WILLIAME Nicolas
ZABAÏOU Rayan
ZHOU Théo
TITON Zélia

Table des matières

Table des figures 3

Table des annexes 4

I. Introduction 6

II. Partie Ops 8

2.1. Mise en place d'un socle de sécurité commun 8

2.1.1. Sécurisation des accès système 8

2.1.2. Filtrage réseau : Configuration du pare-feu Iptables 11

2.2. Mise en place du serveur de production 13

2.2.1. Installation et sécurisation des services applicatifs 13

2.2.2. Exposition sécurisée : Nginx en reverse-proxy 15

2.3. Mise en place du serveur de compilation 17

2.3.1. Préparation de l'environnement système 17

2.3.2. Centralisation des accès : Nginx en reverse-proxy 18

2.3.3. Implémentation de l'ordonnanceur Jenkins 20

2.3.4. Déploiement de la solution d'analyse SonarQube 21

2.3.5. Intégration du gestionnaire d'artefacts Sonatype Nexus 23

2.4. Autres problèmes rencontrés au cours de la partie Ops 26

III. Partie Dev 28
3.1. Base de la Pipeline 28
3.1.1. Configuration des Credentials 28
3.1.2. Tools & Environment 29
3.1.3. Importation du code source depuis Git 29
3.2. Tests version dans Nexus et build 30
3.2.1. Tests de la version de l'artefact dans Nexus 30
3.2.2. Build 30
3.3. Analyse SonarQube et Quality Gate 30
3.3.1. Analyse SonarQube en utilisant SonarScanner 30
3.3.2. Quality Gate & Webhook 32
3.4. Publication dans Sonatype Nexus 32
3.5. Déploiement en production 33
3.6. Stabilisation et débogage de l'infrastructure sur la machine de production 34
3.7. Résultats 35
IV. Conclusion 37
Bibliographie 38
Annexes 42

Table des figures

Figure 1 : Erreur lors du redémarrage du service SSH suite à une mauvaise config 10

Figure 2 : Exemple de commande à utiliser pour se connecter à une VM 11

Figure 3 : Configuration finale du pare-feu Iptables de notre infrastructure 13

Figure 4 : Version de Java installée dans le serveur de production 14

Figure 5 : Création de l'utilisateur Truitteur, son mot de passe et sa base de données .. 15

Figure 6 : Les identifiants définis sur Jenkins 29

Figure 7 : Analyse SonarQube des différents codes source 31

Figure 8 : dépôt Maven Snapshot dans repository Nexus 33

Figure 9 : Cartographie des ports dans les podman 33

Figure 10 : Création d'un compte sur Truitteur 35

Figure 11 : Connexion à son compte Truitteur 36

Figure 12 : Post d'un truitte sur Truitteur 36

Table des annexes

Annexe A – Fichier /home/user/.ssh/Authorized_keys 42

Annexe B – Modifications dans le fichier de configuration du service SSH 43

Annexe C – Fichier /var/log/auth.log 43

Annexe D – Commandes utilisées pour générer les biclés RSA 44

Annexe E – Commandes utilisées pour la configuration du pare-feu Iptables 44

Annexe F – Résultat de la commande « sudo iptables -L –line-numbers » 47

Annexe G – Fichier de configuration d'Nginx dans le serveur de production 48

Annexe H – Fichier de configuration de Sonar /opt/sonarqube/conf/sonar.properties .. 50

Annexe I – Fichier de configuration de Jenkins /var/lib/jenkins/config.xml 52

Annexe J - Fichier de configuration d'Nginx dans le serveur de compilation 54

Annexe K – Fichier de configuration de Nexus nexus.properties 57

Annexe L – Commandes pour la création des Trusstores 57

Annexe M – Création du conteneur Jenkins avec Podman 58

Annexe N – Création du conteneur Nexus avec Podman 58

Annexe O – Création du conteneur PostgreSQL avec Podman 59

Annexe P – Création du conteneur SonarQube avec Podman 59

Annexe Q – Exemple de Pipeline Jenkins 60

I. Introduction

Le projet « Truitteur » est très simple : une application de réseau social existe déjà mais elle n'a nulle part où tourner. Nous avons donc la charge de la déployer et de la sécuriser.

Les spécifications techniques nous indiquent qu'il s'agit d'une application développée sur Java, s'appuyant sur le framework Springboot. L'objectif principal n'est pas le développement de l'application elle-même mais la mise en place d'une infrastructure DevSecOps complète, robuste et sécurisée avec une approche de Security by Design.
Le défi technique réside du fait que l'on commence le projet à zéro, à partir de deux machines virtuelles vierges sur lesquelles tout reste à faire…

Notre mission couvre trois axes majeurs :

1. Opérations (Ops) : Installation et configuration des serveurs (Web, base de données, Application).
2. Sécurité (Sec) : Durcissement des systèmes, gestion des réseaux et analyse des vulnérabilités.
3. Développement (Dev) : Mise en place d'une chaîne d'intégration et de déploiement continus (CI/CD) pour automatiser la livraison du logiciel.

Pour mener à bien ce projet, nous avons créé le groupe Asystes, composé de ZABAÏOU Rayan, ZHOU Théo, WILLIAME Nicolas et TITON Zélia. La première décision à prendre concernait l'hébergement de notre infrastructure. Nous aurions pu utiliser Microsoft Azure ou Google Cloud Platform (GCP) mais nous avons finalement opté pour Amazon Web Services : AWS. Il n'y a pas vraiment de raison, l'offre était intéressante : on a à notre disposition 200$ de crédit, les VM sont disponibles pendant 6 mois, tout cela est largement suffisant pour notre projet et ce choix convenait à tout le monde.

Une fois le serveur de compilation prêt, nous avons travaillé ensemble sur la partie Dev. Chacun travaillait quand il pouvait et faisait part aux autres de ses réussites et/ou problèmes rencontrés, notamment dans un Google Doc commun. Rayan s'est principalement occupé de la mise en forme du rapport sur Word.

Au cours de ce projet, nous avons su mettre en place de manière sécurisée une infrastructure reposant sur deux environnements distincts : un serveur de production et un serveur de compilation.

Nous y avons configuré le service SSH ainsi qu'un pare-feu et la mise en place de Reverse-Proxies Nginx pour sécuriser les accès Web. Nous avons ensuite téléchargé les services nécessaires au bon fonctionnement de l'application.

Le cœur du projet étant la mise en place de la chaîne CI/CD à partir de la rédaction de pipelines Jenkins à travers lesquelles nous avons utilisé divers outils. En effet, nous avons téléchargé des sources via Git, compilé et transformé en artefact le code source importé via Maven et analysé ses vulnérabilités et failles avec les outils SonarQube et SonarScanner. Pour finir, nous avons téléchargé l'artefact avec Sonatype Nexus avant de faire le déploiement sur le serveur de production.

Les principales difficultés que nous avons rencontrées ont été :

De nombreux problèmes liés à l'outil de conteneurisation Podman parmi lesquels :

- Des soucis de permissions des répertoires associés aux conteneurs.

- La communication entre les différents conteneurs.

Enfin, le débogage de l'infrastructure sur la machine de production après le déploiement.

Les principales problématiques soulevées par ce projet ont été :

- Comment bâtir une infrastructure de production résiliente et sécurisée.

- Comment automatiser l'intégralité du cycle de vie de l'application (CI/CD) pour transformer le code source en un service déployé (sans intervention manuelle).

- Comment orchestrer une chaîne d'outils hétérogènes (Jenkins, SonarQube, Nexus) au sein d'une architecture conteneurisée (Podman) tout en assurant leur communication sécurisée.

II. Partie Ops

2.1. Mise en place d'un socle de sécurité commun

2.1.1. Sécurisation des accès système

La première étape critique de notre déploiement a consisté à définir et appliquer une politique de sécurité système commune à l'ensemble de notre infrastructure. Cette procédure de durcissement a été appliquée en simultané sur le serveur de production et sur le serveur de compilation.

En effet, la mise en ligne d'un serveur l'expose aussitôt à des tentatives d'intrusion automatisées. Il est donc primordial de le sécuriser avant d'y installer quoi que ce soit. Les étapes clés de la configuration d'une connexion SSH de manière sécurisée sont :

- L'impossibilité d'accès au compte root,
- L'interdiction de se connecter avec un mot de passe,
- La connexion uniquement par clé SSH.

Nous nous sommes donc renseignés sur les recommandations de l'ANSSI [1], la documentation officielle d'OpenSSH [2], ainsi que diverses ressources techniques et tutoriels partageant les bonnes pratiques à utiliser pour le durcissement de sa configuration et la sécurisation de son serveur SSH [3], [4], [5].

L'usage d'un compte générique comme root pose un problème majeur de sécurité. Nous avons donc mis en place une gestion stricte des identités :

Lorsqu'on lance la VM on commence avec le compte root. On crée tout de suite 4 utilisateurs : Rayan, Theo, Zelia et Nicolas. On crée ensuite le groupe « asystes » auquel on donne les droits sudo à partir de la commande visudo. On y place nos 4 utilisateurs. Via la configuration sudoers, seul le groupe « asystes » est autorisé à utiliser des commandes root.

Bien qu'il soit interdit de se connecter au compte avec un mot de passe, il est primordial pour chaque utilisateur d'en avoir un, et qu'il soit fort, car il sera demandé pour utiliser les commandes administrateurs via sudo. Ainsi, chaque membre du groupe crée son mot de passe en suivant les recommandations de l'ANSSI, à savoir plus de 12 caractères, l'utilisation de caractères spéciaux et une structure aléatoire [6].

Avant de configurer et de durcir le service SSH, chaque membre du groupe doit générer sur son propre pc une biclé RSA qui lui permettra de se connecter à la VM sans utiliser de mot de passe. Pour ce faire, on utilise la commande keygen d'OpenSSH. Ainsi, chaque membre du groupe a sa clé privée RSA enregistrée dans son pc personnel.

On se partage ensuite notre clé publique pour mettre en place une stratégie de redondance d'accès. Techniquement, pour qu'une connexion SSH soit acceptée par le serveur, il faut impérativement que la clé publique du client figure dans le fichier */home/« utilisateur »/.ssh/authorized_keys* du compte cible [1], [3]. Côté client, via PowerShell, la connexion s'effectue avec l'option -i de la commande ssh à laquelle on précise le chemin local vers la clé privée correspondante.

C'est sur ce fonctionnement que repose notre stratégie de redondance : nous avons concaténé les clés publiques de l'ensemble de l'équipe dans le fichier *authorized_keys* de chaque utilisateur. Concrètement, le serveur valide la connexion si la clé privée présentée avec l'option -i correspond à l'une des clés publiques présentes dans le fichier. Cela permet à chaque membre de se connecter au compte d'un collègue avec sa propre clé privée. Cette mesure garantit que même en cas de perte d'une clé ou d'indisponibilité d'un des administrateurs, un autre membre peut légitimement accéder au compte pour intervenir.

Maintenant qu'il est possible pour chaque membre du groupe de se connecter à la VM via les clés SSH, on peut commencer la configuration et le durcissement de ce service. Pour ce faire, nous devons toucher à un fichier qui se trouve dans le dossier */etc/ssh*. C'est ce répertoire qui contient toutes les clés cryptographiques du serveur ainsi que les fichiers de configuration du client et du serveur OpenSSH. Le démon, fichier qui régit le comportement du serveur, est nommé *sshd_config*. Selon la documentation officielle d'OpenSSH [2], il est lu par le service au démarrage pour changer ses paramètres, c'est donc ici que nous avons appliqué nos directives :

PermitRootLogin no → Permet de bannir la connexion directe avec le compte super-utilisateur root. Les bots et attaquants tentent souvent de se connecter avec « admin », « ubuntu » mais surtout « root ». On supprime donc la cible principale d'un potentiel attaquant afin de réduire la surface d'attaque [1], [4], [5]. Par ailleurs, le fichier sshd_config suggère de définir *PermitRootLogin* à « prohibit-password » mais nous avons décidé de le définir à « no » afin d'assurer qu'il soit impossible de se connecter au compte root, même via clé SSH.

PasswordAuthentication no → On désactive l'authentification SSH par mot de passe. Les bots ou attaquants qui essayent d'utiliser un mot de passe usuel comme admin, azerty, root ou quoi pour une attaque brute-force voient leur tentative de connexion échouer immédiatement [1], [3], [5].

ClientAliveInterval 600 [4] → Toutes les 600 secondes, le serveur envoie un paquet « keep-alive » au client. Si le client répond la connexion continue, mais si 3 requêtes restent sans réponse le serveur considère cette connexion comme morte. Cela permet principalement d'éviter d'avoir des sessions inactives toujours activées.

AllowGroups asystes → Seuls les utilisateurs appartenant au groupe « asystes » peuvent se connecter via SSH. Si l'utilisateur n'est pas un membre du groupe, le service coupe immédiatement la connexion [1], [4].

Port 52819 [1], [5] → Il est important de changer le port pour SSH car le port 22 est souvent ciblé par des attaques brute-force. On a donc choisi un autre port pour la connexion SSH. On a décidé d'en prendre un dans la plage des ports dynamiques (49152 - 65535) afin d'éviter les conflits avec d'autres services systèmes. Étant donné que la connexion via mots de passe a été désactivée et que seuls certains utilisateurs sont autorisés, nous n'avons pas à craindre les attaques brute-force mais chaque tentative de connexion est listée dans le fichier /var/log/auth.log [7]. En changeant le port pour SSH, à moins qu'une tentative de connexion ne se fasse sur le port 52819, nous n'aurons pas de tentatives de connexion enregistrées dans les logs, car aucune requête n'atteindra réellement le service SSH.

Un site de tutoriels que nous avons visité suggère également de passer la valeur de la propriété *PermitRootAuthentication* à « no » pour désactiver entièrement l'authentification via le compte root [3]. Toutefois, on a remarqué que cette propriété n'existait pas dans le fichier /ssh/sshd_config à l'origine. Ainsi, lorsque l'on a ajouté la ligne « *PermitRootAuthentication no* » au fichier et redémarré SSH, nous avons eu une erreur de redémarrage du service :

```
rayan@ip-172-31-37-247:~$ sudo vim.tiny /etc/ssh/sshd_config
rayan@ip-172-31-37-247:~$ sudo service ssh restart
Job for ssh.service failed because the control process exited with error code.
See "systemctl status ssh.service" and "journalctl -xeu ssh.service" for details.
```

Figure 1 : Erreur lors du redémarrage du service SSH suite à une mauvaise config

Après quelques recherches, nous avons supprimé cette ligne car la propriété *PermitRootAuthentication* n'est pas mentionnée sur une autre source et est sensiblement identique à *PermitRootLogin*. Nous avons donc considéré qu'il s'agissait d'une erreur du site.

Une fois la configuration terminée, nous avons testé sa validité avec quelques tests d'intrusion. On commence par essayer de se connecter sans préciser le nouveau numéro de port. Toutefois, on a remarqué qu'il était toujours possible de se connecter via SSH. De plus, lorsque l'on précise notre nouveau port avec l'option -p, on ne peut pas se connecter à la VM : Connection timed out. En effet, il fallait préciser le nouveau port d'entrée de la connexion SSH directement depuis le site d'AWS.

On a ensuite essayé de se connecter au compte root avec sa clé RSA, puis de se connecter au compte de chaque membre du groupe avec une mauvaise clé privée. Ces deux tests ont été concluants et confirment que le serveur est désormais hermétique aux attaques basiques et que seuls les membres authentifiés peuvent s'y connecter.

```
PS C:\Windows\system32> ssh -i "C:\Users\Rayan\OneDrive\Documents\4EME ANNEE\P2\Truitteur\
rayan.pem" -p 52819 rayan@13.60.42.168
```

Figure 2 : Exemple de commande à utiliser pour se connecter à une VM

2.1.2. Filtrage réseau : Configuration du pare-feu Iptables

Après avoir sécurisé l'accès au système, la deuxième étape de notre socle de sécurité est la mise en place d'un pare-feu visant à contrôler les flux réseaux entrants et sortants de nos machines. Pour cela, on utilise Iptables, l'outil de filtrage de paquets intégré au noyau Linux.

Tout comme pour la sécurisation du service SSH, cette configuration est appliquée identiquement sur le serveur de production et sur le serveur de compilation.

Pour cette partie, nous avons effectué des recherches sur internet sur la configuration d'un pare-feu avec Iptables, et nous nous sommes notamment appuyés sur la documentation technique d'OVHcloud, qui s'est avérée claire et pertinente pour notre architecture [8].

Le pare-feu se doit d'être le moins permissif possible : par défaut, tout est interdit et on n'autorise explicitement que ce qui est strictement nécessaire au bon fonctionnement de l'application. Ainsi, avant d'appliquer nos règles, on commence par vider les tables existantes pour partir d'une base saine.

La première règle concerne l'interface de boucle locale (loopback). Cette règle est vitale : elle permet aux services interne de communiquer entre eux. Par exemple, Nginx (port 80) parle à Springboot (port 8080), Springboot parle à la base de données PostgreSQL (port 5432), sans jamais exposer ces ports à l'interface publique.

La deuxième règle constitue le suivi de connexion. Elle permet d'accepter automatiquement les paquets entrants qui correspondent à une connexion déjà validée ou initiée par le serveur. Cela évite d'avoir à créer une règle inverse pour chaque requête sortante.

En ce qui concerne les services exposés, nous n'ouvrons que trois portes vers l'extérieur :

- SSH : le port 52819 que nous avons modifié dans la configuration sshd_config et que nous avons ouvert sur le pare-feu externe d'AWS.
- HTTP : le port 80 & HTTPS : le port 443. Ils vont permettre aux utilisateurs d'accéder aux applications (via le serveur web Nginx en reverse-proxy).

À noter que les ports 8080 (Springboot) et 5432 (PostgreSQL) ne sont pas ouverts dans l'INPUT public, ils restent accessibles uniquement en local via la loopback.

Passons maintenant aux règles de sortie. Contrairement à une configuration basique qui laisse tout sortir, nous avons choisi de filtrer également la sortie pour limiter l'impact dans le cas d'une compromission. Nous n'autorisons donc que :

- DNS : le port 53 (UDP/TCP) pour la résolution des noms de domaines. Il sera essentiel pour apt, maven, etc.
- Web sortant : les ports 80 et 443. C'est nécessaire pour les mises à jour système et le téléchargement des dépendances Java par exemple.
- NTP : le port 123 (UDP). Ce service est crucial pour la synchronisation de l'heure ainsi que la validité des logs et des certificats SSL.

Une fois toutes ces règles définies, on applique une politique par défaut restrictive : tout refuser en INPUT, FORWARD et OUTPUT. Nous venons ainsi d'autoriser uniquement les services dont nous aurons besoin, et d'interdire tous les autres.

Il faut ensuite s'assurer que les règles Iptables que nous venons de mettre en place soient persistantes après le redémarrage. Pour ce faire, il faut installer le paquet *iptables-persistent* mais c'est à ce moment que nous avons rencontré un petit problème : la commande *apt update* échouait systématiquement.

En effet, bien que le trafic sortant vers les dépôts Ubuntu fût autorisé, le pare-feu bloquait les paquets de réponse du serveur distant car la règle de suivi de connexion n'était pas prioritaire ou mal positionnée par rapport à la politique par défaut de nos premiers essais manuels. Nous avons donc forcé l'insertion de la règle de suivi d'état en toute première position de la chaîne INPUT. Cela a permis de rétablir la connectivité pour le gestionnaire des paquets. Une fois ce problème résolu, nous avons installé l'outil de persistance d'Iptables et sauvegarder notre configuration.

```
rayan@ip-172-31-33-21:~$ sudo iptables -S
[sudo] password for rayan:
-P INPUT DROP
-P FORWARD DROP
-P OUTPUT DROP
-A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A INPUT -i lo -j ACCEPT
-A INPUT -p tcp -m tcp --dport 52819 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 443 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A OUTPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A OUTPUT -p udp -m udp --dport 53 -j ACCEPT
-A OUTPUT -p tcp -m tcp --dport 53 -j ACCEPT
-A OUTPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A OUTPUT -p tcp -m tcp --dport 443 -j ACCEPT
-A OUTPUT -o lo -j ACCEPT
-A OUTPUT -p udp -m udp --dport 123 -j ACCEPT
```

Figure 3 : Configuration finale du pare-feu Iptables de notre infrastructure

2.2. Mise en place du serveur de production

2.2.1. Installation et sécurisation des services applicatifs

Le serveur de production est la machine critique de l'architecture car c'est ici que l'application finale accessible aux utilisateurs sera hébergée. C'est un peu comme une usine de construction qui se doit d'être stable et minimaliste.

Dans le cadre du projet Truitteur, ce serveur doit uniquement héberger :

- Le moteur d'exécution pour faire tourner le code Java 17
- Le système de gestion de base de données PostgreSQL 15

Pour garantir une installation propre et optimisée sur notre distribution Linux, on s'est appuyés sur la documentation technique d'OVHcloud, notamment leurs guides dédiés à l'installation de Java et PostgreSQL sur Ubuntu [9], [10]. Pour s'assurer de ne pas oublier de détails concernant la sécurisation de ces services et leur bonne installation, nous avons aussi demandé à l'IA de Google, Gemini quelques indications [11].

Ainsi, l'installation de ces services repose sur les principes de minimisation, à savoir n'installer que les paquets strictement nécessaires afin de réduire la surface d'attaque ; et l'isolation de chaque service sous un utilisateur dédié.

On commence bien sûr par mettre à jour les dépôts systèmes avant de commencer nos installations. L'application Truitteur étant développée en Springboot, elle a besoin d'un environnement Java. Nous installons donc le paquet *OpenJDK 17 JRE Headless*. Il s'agit de la version sans interface graphique de l'environnement d'exécution. Contrairement au JDK (qui sert à développer des applications Java), nous n'avons pas besoin des outils de compilation. De plus, la version Headless est plus légère et supprime des vecteurs de vulnérabilités inutiles.

```
nicolas@ip-172-31-33-21:~$ java --version openjdk 17.0.17 2025-10-21
OpenJDK Runtime Environment (build 17.0.17+10-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 17.0.17+10-Ubuntu-124.04, mixed mode, sharing)
```

Figure 4 : Version de Java installée dans le serveur de production

Par défaut, lancer une application avec une compte administrateur est une faille critique car si l'application est compromise, l'attaquant aurait la possession des droits super-utilisateur. Nous avons donc créé un utilisateur système nommé *javauser*, spécifiquement configuré pour être restreint :

- C'est un utilisateur système qui n'a pas de mot de passe
- Il a l'interdiction d'ouvrir un shell
- Personne ne peut se logger sur son compte
- Il ne sert qu'à lancer le processus
- On lui définit son dossier de travail (où résidera l'application)

Pour la persistance des données, le projet requiert PostgreSQL 15. Cependant, les dépôts par défaut d'Ubuntu ne proposent souvent que des versions antérieures. Pour obtenir la version 15 maintenue à jour, nous avons dû ajouter manuellement le dépôt de l'éditeur. Cela implique de télécharger les certificats de sécurité et d'importer la clé de signature GPG officielle pour garantir l'intégrité des paquets téléchargés. Une fois le dépôt ajouté et la liste des paquets mis à jour, on commence l'installation du service.

Étant donné qu'une base de données ne doit jamais être exposée inutilement, on édite le fichier */etc/postgresql/15/main/postgresql.conf* pour y modifier la directive d'écoute sur le localhost. On force donc PostgreSQL à n'accepter que des connexions provenant de la machine elle-même. Ainsi, même si notre pare-feu venait à défaillir, la base de données refuserait toute autre connexion réseau interne.

Enfin, on redémarre le service et on crée un rôle applicatif dédié. On se connecte au compte administrateur *postgres* pour créer :

- Un utilisateur SQL *truitteur* protégé par un mot de passe fort [6].

- Une base de données *truitteur*, dont ce nouvel utilisateur est le propriétaire exclusif. Cela garantit que l'application Java ne pourra accéder qu'à ses propres données et non aux données système du serveur SQL.

```
rayan@ip-172-31-33-21:~$ sudo su - postgres
postgres@ip-172-31-33-21:~$ psql -c "CREATE USER truitteur  WITH PASSWORD '5cn.3&SV1_Mq';"
CREATE ROLE
postgres@ip-172-31-33-21:~$ psql -c "CREATE DATABASE truitteur OWNER truitteur;"
CREATE DATABASE
```

Figure 5 : Création de l'utilisateur Truitteur, son mot de passe et sa base de données

2.2.2. Exposition sécurisée : Nginx en reverse-proxy

Pour exposer l'application, nous avons besoin d'un serveur web Nginx configuré en reverse-proxy. Nginx va agir comme un point d'entrée unique et un bouclier. Il prend en charge la gestion des connexions, la terminaison de chiffrement SSL et le filtrage des requêtes avant de les transmettre à l'application Java qui écoute en local sur le port 8080. Pour la configuration, nous avons suivi les recommandations du tutoriel DigitalOcean [12], ainsi qu'un TP d'un de nos professeurs, Christophe Maudoux, que nous avons eu un peu plus tôt dans l'année [13].

Après l'installation du paquet nginx, nous avons dû gérer la couche cryptographique HTTPS. N'ayant pas encore de nom de domaine public validé par une autorité de certification, nous avons généré un certificat auto-signé via OpenSSL [14].

Avant cela, on commence par vérifier que les ports 80 et 443 sont bien autorisés dans le pare-feu afin qu'Nginx puisse répondre. On crée ensuite deux répertoires différents pour y mettre la clé et le certificat. Une fois le certificat généré, on obtient deux fichiers : la clé publique, et la clé privée.

Tout d'abord, on va faire en sorte que notre machine AWS fasse confiance à nos certificats auto-signés. Pour cela, on va copier notre certificat dans le répertoire */usr/local/share/ca-certificates/* puis on va mettre à jour les certificats avec la commande administrateur *update-ca-certificates*.

Nous avons ensuite créé un fichier dédié */etc/nginx/sites-available/truitteur*. Voici l'analyse détaillée des blocs de sécurité que nous y avons intégrés :

- Une redirection HTTPS forcée. On refuse de servir du contenu non chiffré donc toute requête arrivant sur le port 80 (HTTP) subit une redirection forcée permanente (Code 301) vers le port 443 (HTTPS).

- Le durcissement du service SSL. Dans le bloc serveur 443, on restreint les protocoles utilisés à TLSv1.2 et TLSv1.3. On interdit donc l'usage de vieux protocoles obsolètes et vulnérables comme SSLv3 ou TLS1.0 et TLS1.1. Cela nous garantit que la communication ne peut pas être déchiffrée par des attaques connues (ex : POODLE).

- Protection contre les dénis de service (DoS). On configure des limites strictes pour éviter que le serveur ne soit saturé par des requêtes malveillantes (des Buffer Overflow par exemple). On limite la taille des uploads à 8 Mo pour éviter la saturation du disque/mémoire. On place une limite pour interrompre automatiquement les connexions persistantes après une courte période d'inactivité. On impose aussi un délai maximal pour la réception du corps des requêtes envoyées par les clients. On masque également la version de Nginx pour compliquer la tâche à un attaquant.

- Sur les recommandations de l'OWASP [15], on injecte des en-têtes HTTP spécifiques dans chaque réponse pour protéger le navigateur du client. On empêche le « clickjacking » pour interdire d'afficher notre site dans une iframe externe. On limite aussi les risques d'injections XSS et de MIME-sniffing. Ensuite, on force le navigateur à ne plus jamais tenter de connexion HTTP simple pendant un an.

- Pour finir, la directive *location /* assure le lien avec l'application. Nginx transfère la requête au backend Java tout en portant une attention particulière à la transmission de l'IP réelle du client, sans quoi l'application ne verrait que l'adresse IP 127.0.0.1 (localhost) pour tous les utilisateurs, ce qui empêcherait l'utilisation des logs applicatifs. On a également ajouté une règle sur les fichiers cachés pour empêcher le téléchargement accidentel d'un dossier « .git » oublié par exemple.

Maintenant que l'on a rédigé le fichier de configuration dans le répertoire */sites-available/* il reste une étape pour le rendre effectif, à savoir créer un lien symbolique du fichier vers le répertoire */etc/nginx/sites-enabled/*. Il s'agit d'une convention Debian/Ubuntu : on peut maintenant désactiver le site en supprimant le lien, sans pour autant avoir à enlever le fichier de configuration source. Enfin, on teste la configuration Nginx avant de recharger le service.

Cette configuration avancée d'Nginx met fin à la préparation opérationnelle du serveur de production. En combinant le socle de sécurité qui va de la sécurisation du service SSH au pare-feu Iptables, l'installation cloisonnée des services applicatifs Java 17 et PostgreSQL 15 et ce reverse-proxy durci, nous disposons d'un environnement robuste. La machine est prête à être intégrée dans la chaîne de déploiement continu pour recevoir les premières versions de l'application.

2.3. Mise en place du serveur de compilation

2.3.1. Préparation de l'environnement système

Si le serveur de production est la vitrine de notre application, le serveur de compilation est comme l'atelier de fabrication de cette dernière. Cette machine va nous permettre de transformer le code source brut en une application exécutable, testée et validée.

Contrairement au serveur de production qui se veut minimaliste, celui-ci doit disposer d'une boîte à outils riche pour assurer l'automatisation de la chaîne CI/CD. Concrètement, cette chaîne « Intégration et Déploiement Continus » agit comme une ligne d'assemblage : elle récupère le code des développeurs, le teste et le transforme automatiquement en un produit livrable, en éliminant ainsi les erreurs manuelles.

Conformément au cahier des charges, ce serveur doit héberger les éléments suivants :

- Un environnement de compilation complet : le JDK de Java 17, Maven et Git.
- Un ordonnanceur CI : Jenkins. C'est lui qui pilotera l'ensemble des tâches d'automatisation.
- Une solution SAST : SonarQube. Pour analyser la qualité du code et détecter les failles de sécurité.
- Un gestionnaire d'artefacts : Sonatype Nexus 3. Il sert à stocker les versions compilées de l'application.
- Un serveur Nginx : configuré en reverse-proxy pour exposer de manière sécurisée les différents outils dont on aura besoin.

Contrairement au serveur de production, où un simple environnement d'exécution (la JRE) suffisait, le serveur de compilation nécessite le JDK : Java Development Kit. On installe donc le paquet *openjdk-17-jdk* avec apt. En effet, le JDK contient le compilateur *javac* et les bibliothèques de développement indispensables pour transformer le code source (à savoir les fichiers .java) en bytecode exécutable. Il est essentiel pour permettre à la chaîne CI/CD de construire l'application.

On installe ensuite le paquet *git*. Il s'agit du point d'entrée de notre automatisation. Il va permettre au serveur de se connecter au dépôt distant pour cloner et récupérer le code source.

Ensuite, on télécharge la dernière version stable d'*Apache Maven*. C'est l'outil standard pour l'automatisation de production des projets Java. En plus de compiler, il gère les dépendances en téléchargeant automatiquement les bibliothèques tierces, exécute les tests unitaires et package l'application finale.

2.3.2. Centralisation des accès : Nginx en reverse-proxy

Afin de sécuriser l'accès à notre usine logicielle et de ne pas exposer directement les services applicatifs sur le réseau, nous avons installé et configuré le serveur web Nginx. Son rôle ici est double :

- La sécurité : Il agit comme un point d'entrée unique, permettant de n'ouvrir que les ports standards sur le pare-feu, au lieu d'exposer individuellement Jenkins, SonarQube et Sonatype Nexus.

- La gestion SSL centralisée : Il gère le chiffrement HTTPS pour tous les outils, ce qui nous évite de devoir configurer des certificats sur chaque application individuellement.

Afin de tout mettre en place, on utilise à nouveau le tutoriel de DigitalOcean [12] que l'on a utilisé pour configurer Nginx sur le serveur de production. On utilisera également un tutoriel du site Hostinger [16] afin de nous aider pour les différents blocs du fichier de configuration.

Ce fichier de configuration : */etc/nginx/sites-available/ci-cd* que nous avons mis en place définit les règles de routage pour Jenkins, SonarQube et Sonatype Nexus. *Note : Le détail de l'installation de ces trois outils sera traité spécifiquement dans les parties suivantes (2.3.3 à 2.3.5). Nous nous concentrons ici uniquement sur leur exposition réseau.*

Voici donc l'analyse des blocs techniques clés pour notre configuration [22] :

La gestion des flux et redirection HTTPS → Comme pour le serveur de production, nous avons appliqué une politique de sécurité stricte. Tout d'abord, la redirection 301 : Le premier bloc *server* écoute sur le port 80 et force immédiatement la redirection vers le port 443. Ensuite, l'utilisation de protocoles robustes : Nous avons restreint les protocoles de sécurité à TLSv1.2 et TLSv1.3 et désactivé l'affichage de la version d'Nginx pour limiter la prise d'information par un attaquant potentiel.

Le bloc Jenkins (port 8080) → L'exposition de Jenkins nécessite des réglages fins pour supporter ses fonctionnalités en temps réel. Premièrement, les WebSockets. Les directives *proxy_set_header Upgrade* et *Connection "upgrade"* sont indispensables. Elles permettent à l'interface de Jenkins de se rafraîchir en direct (ex : voir les logs de build défiler) sans recharger la page. Ensuite, en ce qui concerne les performances. Nous avons défini un *upstream* avec *keepalive 32* pour maintenir des connexions ouvertes entre Nginx et Jenkins, réduisant la latence. Enfin, les timeouts. Les délais ont été augmentés à 90 secondes pour éviter que Nginx ne coupe la connexion pendant un build un peu long.

Le bloc SonarQube (port 9000) → La configuration pour SonarQube est plus classique. Nginx redirige les requêtes du contexte */sonar* vers le service local écoutant sur le port 9000. Nous prenons soin de transmettre les en-têtes d'origine pour que SonarQube identifie correctement l'adresse IP des utilisateurs et non celle du proxy local.

Le bloc Nexus (port 8081) et la gestion des fichiers lourds → En tant que gestionnaire d'artefacts, Nexus est amené à recevoir les fichiers très volumineux (les fichiers .jar compilés par exemple). Il faut donc ajouter la directive *client_max_body_size 1G;* En effet, par défaut, Nginx bloque les uploads supérieurs à 1 Mo. Sans cette directive explicite augmentant la limite à 1 Go, tout déploiement de l'application vers Nexus échouerait avec une erreur du style « 413 Request Entity Too Large ».

On teste ensuite cette configuration et on redémarre le service Nginx. Cette configuration permet à notre chaîne CI/CD d'être accessible de manière unifiée et sécurisée derrière une seule adresse IP publique.

2.3.3. Implémentation de l'ordonnanceur Jenkins

Jenkins est un serveur d'automatisation open-source qui agit un peu comme un chef d'orchestre : il détecte les changements dans le code, lance la compilation et effectue les tests.

Pour l'installation de Jenkins, nous avons opté pour une architecture conteneurisée. Cela nous permet d'isoler chaque outil, de faciliter les mises à jour et de garantir que l'environnement est réplicable.

Cependant, au lieu d'utiliser Docker, nous avons opté pour **Podman** [32]. En effet, il s'agit d'une architecture « Daemonless » : contrairement à Docker qui repose sur un démon central (un processus qui tourne en permanence avec les droits root), Podman possède un mode Rootless (sans les privilèges root). Il lance les conteneurs comme des processus enfants directs. Cela améliore à la fois la stabilité et surtout la sécurité du système. De plus, Podman est compatible avec les images OCI (Open Container Initiative), ce qui nous permet d'utiliser les images Docker officielles sans modifications [17].

Nous avons donc récupéré l'image officielle incluant le JDK 17 de Jenkins à partir de leur site internet [18]. Pour lancer le service, nous avons utilisé une commande *podman run* spécifique intégrant plusieurs paramètres critiques pour notre infrastructure [21] :

- **Un réseau dédié** : Le conteneur est attaché au réseau privé devops-net que nous avons créé au préalable. Cela permettra à Jenkins de communiquer directement avec SonarQube et Nexus par leurs noms de conteneurs.

- **Exposition des ports** : 8080 : Pour l'interface web (UI) et 50000 : Pour la communication avec les agents de build (JNLP).

- **Persistance des données** : Les conteneurs étant éphémères (les données disparaissent si on les redémarre), nous avons monté un volume liant le dossier hôte *~/jenkins_data* vers */var/jenkins_home* dans le conteneur.

- **Adaptation au reverse-proxy Nginx** : C'est un point de configuration crucial. Comme Jenkins sera accessible via *https://truitteur.../jenkins* (et non à la racine du domaine), nous avons dû forcer le préfixe d'URL au démarrage via la variable d'environnement : *--env JENKINS_OPTS="--prefix=/jenkins"*. Sans cette option, l'application répondrait avec des liens absolus (ex: */login* au lieu de */jenkins/login*), brisant la navigation derrière Nginx [20].

En tentant le lancer le conteneur, nous avons aussi rencontrés quelques problèmes techniques, notamment de permissions (UID/GID).

Lors du premier lancement, le conteneur a immédiatement échoué avec des erreurs de type « Permissions Denied ». L'image Docker de Jenkins est configurée pour exécuter le processus avec un utilisateur interne spécifique ayant l'identifiant (UID) 1000. Or, le dossier de persistance que nous avons créé sur la machine hôte (*~/jenkins_data*) appartenait à notre utilisateur système ou root. Le conteneur n'avait donc pas les droits d'écriture nécessaires pour initialiser ses fichiers de configuration [19].

Il a donc fallu harmoniser les droits entre l'hôte et le conteneur. Nous avons identifié l'UID requis par l'image (1000 pour Jenkins) et avons appliqué manuellement les droits sur le dossier de l'hôte via la commande : *sudo chown -R 1000:1000 ~/jenkins_data*. Cette manipulation a permis au conteneur d'écrire correctement ses données.

Une fois le service démarré, la configuration interne de Jenkins est stockée dans des fichiers XML (notamment /var/lib/jenkins/config.xml). Ces fichiers, définissent les paramètres de sécurité, le nombre d'exécuteurs de build et la stratégie de nommage des projets. Nous n'avons pas eu à les modifier manuellement, l'interface graphique se chargeant de les générer.

2.3.4. Déploiement de la solution d'analyse SonarQube

Au cœur de notre démarche DevSecOps, nous intégrons SonarQube, une plateforme d'analyse de code statique (SAST) [26]. Cet outil sert à inspecter le code source en continu pour détecter des bugs, des mauvaises pratiques et surtout des failles de sécurité avant même que l'application ne soit déployée. C'est l'application concrète du principe de « Security by Design ».

Tout comme pour Jenkins, nous n'avons pas installé SonarQube directement sur la machine, mais sous forme de deux conteneurs interconnectés via le réseau privé *devops-net*.

Le premier d'entre eux est la base de données : *sonar-db*. SonarQube nécessite une base de données pour stocker ses règles et les rapports d'analyse. Nous avons donc utilisé une image PostgreSQL :

- **Configuration** *:* Variables d'environnement définies pour l'utilisateur (*sonar*), le mot de passe et la base de données.

- **Volume** *:* Persistance des données via un mappage du dossier hôte *~/sonar_db* vers le conteneur.

Le second correspond au serveur applicatif : nous avons déployé la version « TLS Community » de SonarQube.

- **Liaison BDD** *:* Connexion configurée via la variable *SONAR_JDBC_URL* pointant vers le conteneur [23].

- **Contexte Web** *:* Configuration essentielle pour le Reverse Proxy, nous avons forcé l'application à répondre sur */sonar* (et non à la racine) via *SONAR_WEB_CONTEXT=/sonar* [23].

Comme pour Jenkins, nous avons rencontré des problèmes techniques. Au premier démarrage, les conteneurs ont échoué (CrashLoopBackOff). Il y avait deux problèmes :

Problème Elasticsearch → Le moteur de recherche intégré à SonarQube plantait avec une erreur « AccessDeniedException » sur le fichier *node.lock*. La cause était que le processus interne de SonarQube s'exécute avec l'UID 1000, mais le dossier de persistance sur l'hôte appartenait à root. Nous avons donc réattribué la propriété du dossier via *chown* [25].

Problème PostgreSQL → La base de données refusait de démarrer (Permission denied sur *pg_filenode.map*). Nous avons dû élargir les permissions dur le volume de base de données pour contourner les restrictions strictes de l'image PostgreSQL sous Podman.

L'exposition de SonarQube derrière Nginx a aussi soulevé un problème de configuration complexe. Après avoir configuré le bloc *location /sonar* dans Nginx, nous obtenions systématiquement une erreur 404 ou une page par défaut Tomcat lors de l'accès via *https://truitteur.../sonar/*.

En analysant les logs et en testant avec *curl*, nous avons compris que Nginx transmettait bien la requête, mais que le chemin n'était pas correctement interprété par l'application Java backend. Le proxy_pass pointait vers http://127.0.0.1:9000/, omettant le contexte. Nous avons corrigé la directive proxy_pass pour inclure explicitement le sous-dossier : proxy_pass *http://127.0.0.1:9000/sonar/;* Cette modification, couplée à l'ajout des en-têtes X-Forwarded-Proto (pour gérer le HTTPS) et X-Real-IP, a permis d'accéder correctement au tableau de bord.

Une fois le serveur fonctionnel, nous avons configuré la partie « client » qui effectue l'analyse du code. Nous avons donc mis en place l'analyseur SonarScanner [24]. Pour ce faire, nous sommes passés par trois grandes étapes :

Création du Projet → Sur l'interface web, nous avons créé le projet « Truitteur » (clé : Truitteur-key) et généré un jeton d'accès sécurisé (Truitteur-token) pour éviter de stocker des mots de passe en clair dans nos scripts.

Installation du Scanner CLI → Nous avons téléchargé et extrait l'archive du SonarScanner dans */opt/sonar-scanner*. Pour rendre l'outil accessible globalement (sans devoir taper le chemin complet), nous l'avons ajouté au PATH système en modifiant le fichier */etc/profile* : *export PATH=$PATH:/opt/sonar-scanner/bin*.

Lancement d'une analyse → La connexion entre le scanner et le serveur a nécessité une dernière adaptation. Le certificat SSL étant auto-signé, le scanner Java refusait initialement la connexion. Nous avons dû importer notre certificat dans le magasin de confiance (Truststore) de Java ou configurer le scanner pour accepter les certificats locaux. L'analyse est ensuite lancée via la commande sonar-scanner, en précisant l'URL du projet et le token d'authentification [27].

2.3.5. Intégration du gestionnaire d'artefacts Sonatype Nexus

Dernier maillon de notre infrastructure, Sonatype Nexus Repository Manager agit comme l'entrepôt de stockage de notre usine logicielle. Il remplit deux fonctions critiques pour le projet Truitteur :

- **Proxy de dépendances** → Il met en cache les bibliothèques téléchargées depuis Internet (Maven Central), accélérant les builds futurs.

- **Dépôt de livrables** → Il stocke les versions compilées et validées de notre application (fichiers .jar), permettant de déployer une version spécifique sans avoir à la recompiler.

À la base, nous avions opté pour une installation manuelle de Nexus [28]. Par mesure de sécurité, il est proscrit de faire tourner un tel service en tant que root, nous avons commencé par créer un utilisateur système dédié : nexus. Nous avons fait en sorte qu'il soit impossible de se connecter à l'utilisateur système et qu'il n'ait pas de mot de passe par soucis de sécurité.

L'étape suivante consistait à récupérer les sources de l'application.

C'est ici que nous avons rencontré notre premier obstacle technique. Nous avons initialement tenté de télécharger l'archive via le lien générique souvent cité dans les documentations [29].

Cette commande s'est soldée par une erreur 404 Not Found. Il semblerait que l'éditeur ait modifié la structure de ses liens ou supprimé l'alias "latest". Nous avons dû nous rendre manuellement sur le site officiel de Sonatype pour identifier l'URL précise de la version stable actuelle [30]. Nous avons ainsi pu télécharger la version 3.87.1 dans le dossier temporaire */tmp*.

Une fois l'archive extraite (*tar -xvf*), nous avons obtenu deux dossiers distincts que nous avons organisés rigoureusement dans */opt/nexus* :

- **Le dossier applicatif (nexus-3.87.1)** → Renommé et déplacé vers */opt/nexus/app*. Il contient le binaire et les bibliothèques logicielles.

- **Le dossier de données (sonatype-work)** → Déplacé vers */opt/nexus/data*. Il contient la base de données, les logs et les artefacts stockés.

Cette séparation physique est cruciale pour faciliter les futures mises à jour sans risquer d'écraser les données. Enfin, nous avons transféré la propriété de l'ensemble de ces dossiers à l'utilisateur système créé plus tôt avec *chown*.

Pour garantir que Nexus démarre automatiquement au boot du serveur et puisse être redémarré en cas de plantage, nous ne l'avons pas lancé manuellement. Nous avons créé un service système dans */etc/systemd/system/nexus.service*.

Ce fichier déclare l'utilisateur nexus comme propriétaire du processus, définit le chemin vers l'exécutable et configure les délais de démarrage. L'activation s'est faite via les commandes standard qui permettent la prise en compte du nouveau fichier, l'activation au démarrage et le lancement immédiat.

Finalement, dans un souci d'uniformisation avec Jenkins et Sonar, nous avons déployé Nexus sous forme de conteneur Podman. Cela facilite la maintenance et l'intégration au réseau privé devops-net déjà en place. Nous avons donc utilisé l'image officielle de *sonatype/nexus3*. Voici les choix techniques de notre configuration :

- **Context Path (-e)** → La variable *NEXUS_CONTEXT_PATH=/nexus* est cruciale. Elle force l'application à répondre sur le sous-dossier /nexus (et non à la racine), ce qui est indispensable pour qu'elle fonctionne correctement derrière notre Reverse Proxy Nginx. Cette option extrait l'image du registre Docker public.

- **Persistance (-v)** → Nous avons monté un volume hôte *(~/nexus_data*) pour garantir que les artefacts et la configuration survivent au redémarrage ou à la mise à jour du conteneur.

Le déploiement conteneurisé a présenté deux défis spécifiques qu'il a fallu résoudre.

Le premier est un conflit de permissions (UID 200) : Au premier lancement, le conteneur a immédiatement crashé avec une erreur de type *Access Denied* sur le volume de données. L'image Docker de Nexus est configurée pour utiliser l'utilisateur interne avec l'UID 200 [30]. Or, le dossier créé sur notre machine hôte *(~/nexus_data*) appartenait à notre utilisateur courant (UID 1000). Le conteneur ne pouvait donc rien y écrire. Nous avons harmonisé les droits en attribuant la propriété du dossier hôte à l'UID spécifique de Nexus avec *chown*.

Le second est la récupération du mot de passe initial : Contrairement à une installation classique où l'on définit le mot de passe, Nexus génère un secret aléatoire au premier démarrage stocké dans un fichier interne temporaire. Pour se connecter la première fois, nous avons dû lire ce fichier directement dans le conteneur via la commande : *podman exec nexus cat /nexus-data/admin.password*. Une fois connecté avec ce token, nous avons défini un mot de passe administrateur et désactivé l'accès anonyme pour sécuriser nos dépôts.

Le problème est que nous ne savions pas que ce fichier était temporaire. Nous avons donc perdu le mot de passe la première fois et avons dû supprimer le répertoire nexus-data ainsi que le conteneur Podman Nexus. Nous avons ensuite tout regénéré et ainsi récupéré le fichier contenant le mot de passe temporaire [31].

L'accès final se fait via le Reverse Proxy (configuré en partie 2.3.2). Le point de vigilance majeur ici a été la directive *client_max_body_size 1G*. Sans cette augmentation explicite de la taille des paquets autorisés dans Nginx, tout tentative d'upload d'un artefact Java (fichier .jar) par notre pipeline CI/CD aurait été rejetée par une erreur « 413 Payload Too Large ».

Avec la finalisation de cette étape, le Serveur de Compilation est désormais pleinement opérationnel. Nous avons réussi à mettre en place une "Usine Logicielle" moderne et entièrement conteneurisée grâce à Podman :

- **L'Approvisionnement** → Git pour les sources.
- **La Construction** → JDK 17 et Maven pour la compilation.
- **L'Orchestration** → Jenkins.
- **Le Contrôle Qualité** → SonarQube et sa base PostgreSQL.
- **Le Stockage** → Nexus Repository Manager.
- **L'Accès unifié** → Nginx sécurisant l'ensemble via HTTPS et un routage par URL.

Tous les services communiquent de manière isolée sur le réseau interne *devops-net*, garantissant sécurité et performance. Nous pouvons désormais passer à la phase suivante : la création des Pipelines CI/CD qui vont automatiser le flux de travail des développeurs.

2.4. Autres problèmes rencontrés au cours de la partie Ops

Au cours de notre travail dans cette partie Ops, nous avons rencontré plusieurs problèmes dont certains étaient trop anodins pour être traités dans les grandes parties, mais suffisamment intéressants pour être expliqués. C'est donc ici qu'ils seront traités, en suivant l'ordre des parties.

Tout d'abord, lors de la première séance, nous n'avions pas utilisé une bonne méthode pour générer les clés privées RSA. Une seule personne en a générées 4 sur un PC de l'école et les a distribuées aux autres membres du groupe, ce qui pose un souci de confidentialité et donc de sécurité. Nous avons donc par la suite décidé de générer chacun notre biclé RSA et de se transmettre nos clés publiques uniquement, ce qui est la bonne pratique à faire.

De plus, la première fois que nous avons configuré les règles SSH dans le fichier de configuration du service, nous avons bien ajouté nos utilisateurs au groupe des sudoers et rendu impossible la connexion au compte root. Le problème est que nous avions oublié de mettre en place un mot de passe aux utilisateurs. Ainsi, connectés à la VM avec nos propres comptes nous ne pouvions plus utiliser les commandes super-utilisateurs. L'accès au compte root via SSH était rendu impossible et on ne pouvait pas non plus via la ligne de commande car il fallait les droits admin… Nous avons donc supprimé la VM (qui ne contenait presque rien) et recommencé depuis le début.

La première fois que nous avons ramené nos PC portables à l'école pour travailler avec, nous n'arrivions pas à nous connecter à la VM et il nous a fallu une heure avant de découvrir que la connexion internet de l'ESIEE bloquait l'accès à la VM d'AWS. Il nous a donc fallu nous connecter avec un partage de connexion depuis notre téléphone pour travailler sur nos machines.

De plus, lorsque Théo se renseignait sur l'installation de Jenkins, il a découvert que le site officiel [17] recommandait des ressources minimums à avoir sur la machine, à savoir 8 Go de RAM. C'est là que nous avons découvert que notre VM n'en avait que 0.9 à disposition. Heureusement, il a été possible de le modifier facilement depuis le site d'AWS et nous n'avons pas eu à tout recommencer à cause de ce souci.

III. Partie Dev

3.1. Base de la Pipeline

3.1.1. Configuration des Credentials

Nous devons désormais réaliser 5 pipelines afin de mettre en place l'intégration et le déploiement continus (CI/CD) : à savoir la récupération du code source de l'application, sa compilation, son inspection avec SonarQube, sa publication en tant qu'artefact dans Nexus et son déploiement sur la machine de production que nous avons mise en place.

Cette section détaille la construction de notre pipeline « Authentication-Service » qui s'occupe du code source du repository « Authentication ». Les autres pipelines sont similaires à celle-ci à l'exception de celle exploitant le repository « Angular-Front ». En effet, cette dernière présente du code en JavaScript et non en Java.

Pour commencer, dans Jenkins, nous créons un fichier *jenkinsfile* de type jenkins.

- **Accès à SonarQube (SONAR_TOKEN)** → Un *Secret Text* contenant le jeton d'authentification généré par SonarQube. Ce token remplace l'authentification par mot de passe et permet au **SonarScanner** de soumettre les rapports d'analyse sans exposer les identifiants administrateur.

- **Accès au Dépôt de Code (git-auth)** → Identifiants de type *Username with Password* permettant à Jenkins de cloner le dépôt Git privé hébergé sur le serveur GitLab/Gitea de l'école.

- **Accès à Nexus (NEXUS_CREDS)** → Identifiants administrateur permettant à Maven de se connecter au dépôt maven-snapshots pour publier l'artefact .jar intermédiaire et vérifier l'existence des versions précédentes.

- **Accès à la machine de prod (PROD_CREDS)** → Identifiants et clé privée SSH permettant d'accéder à la machine de production.

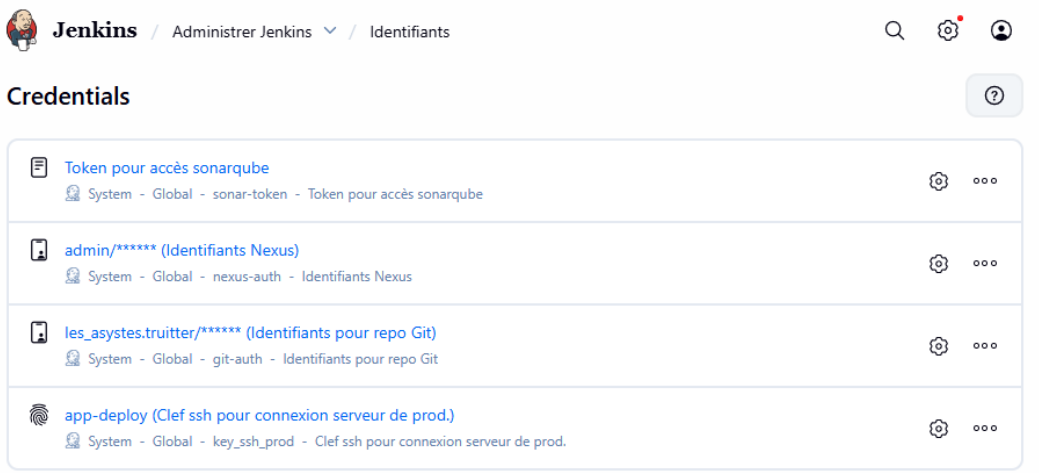

Figure 6 : Les identifiants définis sur Jenkins

3.1.2. Tools & Environment

Nous pouvons désormais l'intégrer au bloc « Environment » du pipeline.

Pour garantir la reproductibilité des builds, nous définissons les outils nécessaires dans les blocs « Tools » : Maven 3 et JDK 17. À préciser que ces deux outils ont été nommés et installés via des plugins et le système de gestion d'outils de Jenkins [33], [36].

Comme précisé plus haut dans le rapport, JDK 17 est la 17e version du kit de développement Java. En ce qui concerne Maven, il s'agit d'un gestionnaire de dépendances et de cycle de vie du build (compilation, déploiement d'artefacts, …).

Une spécificité du pipeline d'Angular-front est qu'on doit utiliser l'outil NodeJS à la place de Maven et de la JDK 17. Toutefois, la version d'OpenSSL utilisée par les dépendances de NodeJS était probablement trop ancienne par rapport à la version dans notre NodeJS, ce qui causait une erreur. Ainsi, nous avons ajouté une ligne *--ssl -legacy -provider* dans le bloc « Environment » afin de rétablir la compatibilité.

3.1.3. Importation du code source depuis Git

À présent, nous devons importer le code source de l'application Truitteur, stocké dans un repository GitHub. Pour ce faire, nous allons préciser dans quelle branche du GitHub nous allons le chercher « Master » puis nous précisons quels identifiants utiliser (ceux que nous avons définis dans le bloc « Environment »). Enfin, nous pouvons définir l'URL d'importation du type « https://gitgud.totouiproject.ovh/Cours_ESIEE/code/code_source.git ».
Note : dans cette URL type, il faut remplacer « code_source » par le nom de repository voulu.

3.2. Tests version dans Nexus et build

3.2.1. Tests de la version de l'artefact dans Nexus

Ici, nous allons chercher à faire en sorte que notre pipeline ne reconstruise pas inutilement un artefact déjà existant. Pour cela, avant toute compilation, on interroge notre dépôt Nexus.

On commence par l'extraction de la version avec la commande *mvn help:evaluate* qui extrait la version déclarée dans le pom.xml du code source. Le pipeline vérifie l'existence de l'artefact en construisant l'URL théorique de ce dernier et en effectuant une requête HTTP avec la commande *curl* vers Nexus.

Pour finir, si le code HTTP est 200 (ok) et que la version est une « RELEASE », la variable « BUILD_NEEDED » passe à *false* et le build est ainsi annulé, car non nécessaire. Toutefois, si la version est introuvable ou qu'il s'agit d'une « SNAPSHOT », la variable « BUILD_NEEDED » est passée à *true* et le build a donc lieu [35].

3.2.2. Build

La première chose que vérifie le bloc « Build & Test » est la valeur de la variable « BUILD_NEEDED ». En effet, si cette dernière est à *true*, alors la commande *mvn clean package -DskipTests* est exécutée. Celle-ci permet de nettoyer l'espace de travail, compiler le code source et générer l'archive jar dans le dossier *target/*.

Spécificités du frontend Angular : il produit des fichiers statiques (et donc pas un exécutable Java). Ainsi, nous avons dû adapter le build avec un passage de Maven à NPM en utilisant la commande *nmp run build –prod* pour la « transpilation ».

3.3. Analyse SonarQube et Quality Gate

3.3.1. Analyse SonarQube en utilisant SonarScanner

Dans cette partie, nous allons vérifier la qualité du code. Nous allons donc utiliser l'outil SonarScanner configuré en tant que plugin Jenkins. Une fois de plus, on vérifie que la valeur de la variable « BUILD_NEEDED » est bien à *true*. L'étape est encapsulée dans le bloc *withSonarQubeEnv('sonar-server')*, ce qui permet d'injecter automatiquement les variables de connexion au serveur SonarQube [37].

Le scanner est lancé avec des paramètres spécifiques, tels que la clé du projet, l'emplacement des fichiers sources et binaires et les identifiants et l'URL de SonarQube.

Cette analyse va permettre de faire remonter les bugs, vulnérabilités, Code Smells, etc. vers le Dashboard SonarQube.

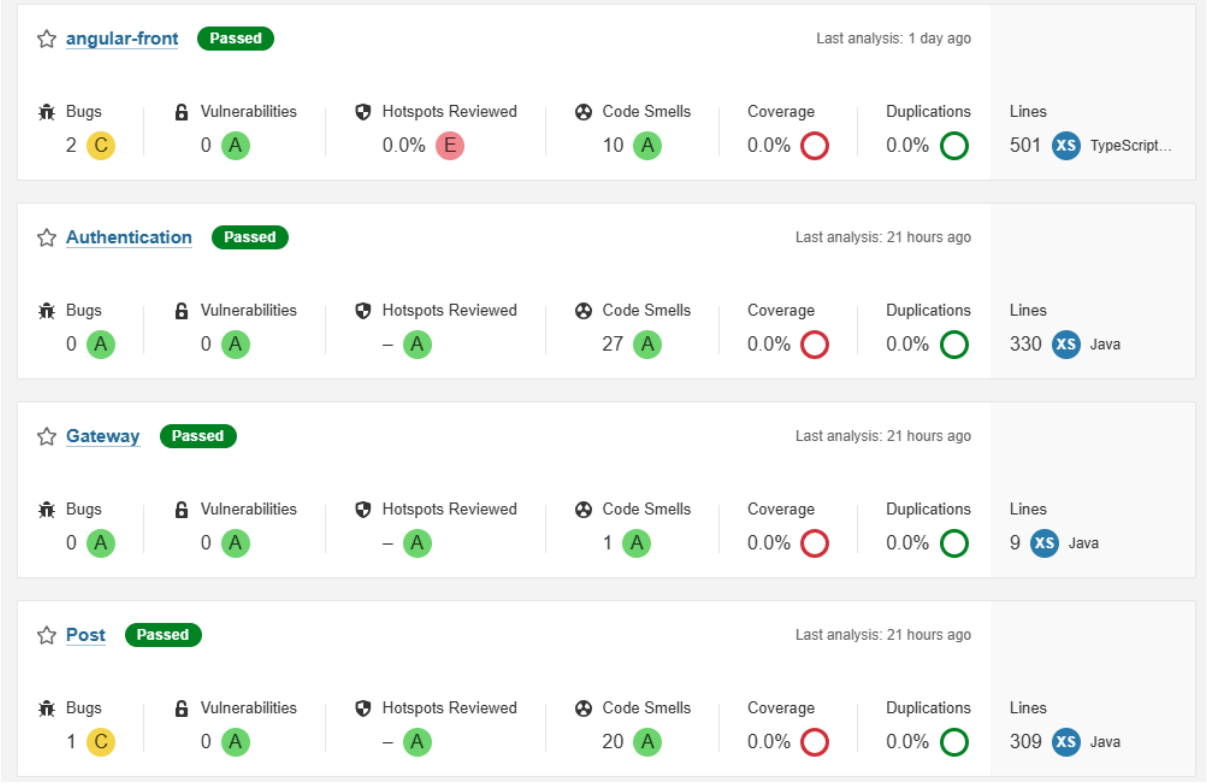

Figure 7 : Analyse SonarQube des différents codes source

À préciser que nous avons rencontré une erreur au niveau de la communication de l'analyse SonarScanner dans Jenkins vers SonarQube (HTTP 404). Cette erreur était due au fait que nous accédions à SonarQube via l'URL *http://sonarqube:9000/sonar*.

Le */sonar* n'est pas par défaut et Jenkins ne trouvait donc pas SonarQube. Nous avons dû le rajouter dans la configuration système de Jenkins pour SonarQube servers.

Afin de trouver l'erreur, nous avons créé une nouvelle étape dans notre pipeline : on faisait des *curl* vers SonarQube et son API.

3.3.2. Quality Gate & Webhook

Lorsque Jenkins envoie le code, SonarQube prend un certain temps pour le traiter. Ainsi, pour éviter que Jenkins ne boucle inutilement en demandant le statut, nous utilisons un mécanisme de notifications inverse (SonarQube → Jenkins) via un Webhook [40].

Du côté de Jenkins, on utilise la fonction *waitForQualityGate* qui met le job en pause jusqu'à réception d'un signal de SonarQube indiquant la fin du test.

Du côté de SonarQube, on peut configurer une Quality Gate en précisant nos paramètres. Pour notre cas, nous avons gardé le default. Nous avons configuré un Webhook pointant vers l'URL interne de Jenkins *http://jenkins:8080/jenkins/foobar/sonarqube-webhook/*.

Nous avons rencontré plusieurs problèmes au cours de cette partie.

Premier problème : le blocage infini. SonarQube finissait l'analyse mais Jenkins restait en attente indéfiniment. Ceci était dû à un problème de résolution DNS. En effet, le conteneur SonarQube ne parvenait pas à contacter le conteneur Jenkins via localhost. Ainsi, nous avons dû modifier l'URL du Webhook pour utiliser le nom du service Docker (*/jenkins/foobar*). De plus, nous avons ajouté un timeout (une fonction) permettant de bloquer l'échec en cas de blocage.

Deuxième légère erreur : Jenkins recevait le signal mais rejetait la Payload (Erreur 403/401). Cela venait simplement d'une mauvaise configuration du token d'authentification dans Jenkins.

3.4. Publication dans Sonatype Nexus

Une fois le code validé par la Quality Gate (selon ses paramètres), l'artefact est publié dans le gestionnaire de dépôts. Pour cela, on utilise une commande *curl* pour effectuer un upload HTTP vers le dépôt maven-snapshots de Nexus. Cette étape permet d'assurer que chaque version déployée en production est sauvegardée et disponible pour des rollbacks éventuels [35], [38].

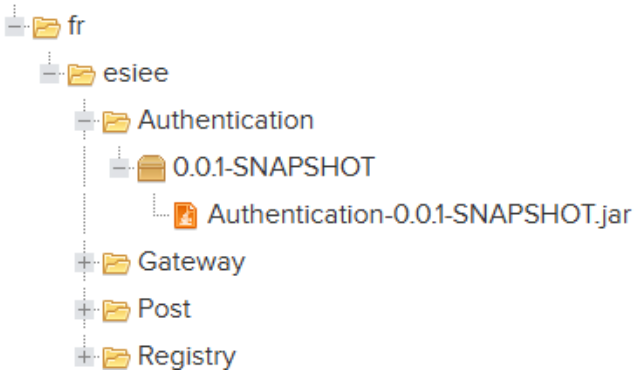

Figure 8 : dépôt Maven Snapshot dans repository Nexus

3.5. Déploiement en production

On commence la phase de déploiement continu de notre pipeline qui consiste à déployer l'application sur le serveur de production à l'aide de conteneurs Podman en Rootless (pour plus de sécurité). Le défi principal est que plusieurs services Java écoutent par défaut sur le même port (8080). Or, nous ne possédons qu'une seule machine. Ainsi, nous avons décidé d'utiliser des conteneurs indépendants avec un mappage de ports strict différent entre l'intérieur et l'extérieur du conteneur (pas de pods afin d'éviter l'erreur *bind: address already in use*).

Service	Port Interne (Conteneur)	Port Hôte (Externe)	Justification
Gateway	8080	**8080**	Point d'entrée unique public.
Authentication	8080	**8081**	Accessible uniquement via le Gateway.
Post/Tweets	8080	**8082**	Accessible uniquement via le Gateway.
Front Angular	80	**3000**	Serveur Nginx statique.
Discovery	8761	**8761**	Serveur Eureka standard.

Figure 9 : Cartographie des ports dans les podman

Rentrons dans la pratique. Nous allons chercher à accéder à la machine de production depuis notre machine de compilation afin d'y acheminer l'artefact (fichier .jar). Pour cela, nous utilisons une connexion SSH utilisant les identifiants définis précédemment dans le rapport pour afin de créer des répertoires nécessaires au déploiement de l'application. Nous utilisons ensuite SCP (toujours avec les mêmes identifiants) pour acheminer l'archive.

Par la suite, nous nous connectons de nouveau à la machine de production. On y arrête et supprime le conteneur précédemment instancié au même nom (s'il existe). On démarre ensuite un nouveau conteneur pour le service utilisant le code source de l'archive.

Nous aborderons les spécificités de chacun des conteneurs pour les différents services de l'application dans la partie suivante.

3.6. Stabilisation et débogage de l'infrastructure sur la machine de production

Une fois les pipelines fonctionnels, nous avons dû stabiliser l'interaction entre les différents services de l'application.

Premier problème : connectivité à la base de données (*UnknownHostException*).

Nos services backend « Authentication », « Gateway » et « Post » doivent avoir accès à la base de données PostgreSQL. Cependant, ils ne parvenaient pas à la trouver. Pour régler ce problème, nous avons précisé au lancement du conteneur podman l'adresse d'où était notre BDD (qui ne tourne pas dans un podman) ainsi que le nom d'utilisateur et le mot de passe à utiliser [39].

Deuxième erreur : les services n'arrivaient pas à communiquer entre eux.

Ils ne se trouvaient pas car ils cherchaient sur leur propre localhost. Pour régler ce problème nous avons créé, comme dans le serveur de compilation, un réseau *esiee-network*.

Troisième problème : le frontend Angular. On avait une erreur de routage 404. Nous avons donc dû configurer le serveur web Nginx pour rediriger toutes les requêtes vers index.html via la ligne *try_files \$uri \$uri/ /index.html;*. Cela permet à Angular de gérer le routage SPA (Single Page Application) lors d'un rafraîchissement de page.

Quatrième problème : l'enregistrement des services dans l'annuaire « Eureka » du service « Registry » a posé quelques problèmes… [34]

En effet, les services Springboot s'enregistraient avec leur hostname, souvent résolu comme localhost à l'intérieur du conteneur. Ainsi, la gateway tentait de rediriger le trafic vers localhost:8081 (c'est-à-dire elle-même) où aucun service n'écoutait. Cela causait une erreur « Connection Refused » C'est pourquoi nous avons forcé l'utilisation de l'IP du conteneur via la variable d'environnement *EUREKA_INSTANCE_PREFER_IP_ADDRESS=TRUE* qui permet aux services de s'enregistrer avec leur IP routable et non plus avec leur hostname. Après avoir redémarré la gateway (en faisant attention de la lancer après que les autres services s'étaient enregistrés), elle pouvait rooter le trafic correctement à travers le réseau podman.

3.7. Résultats

Notre application Truitteur peut désormais tourner correctement, en voici les résultats :

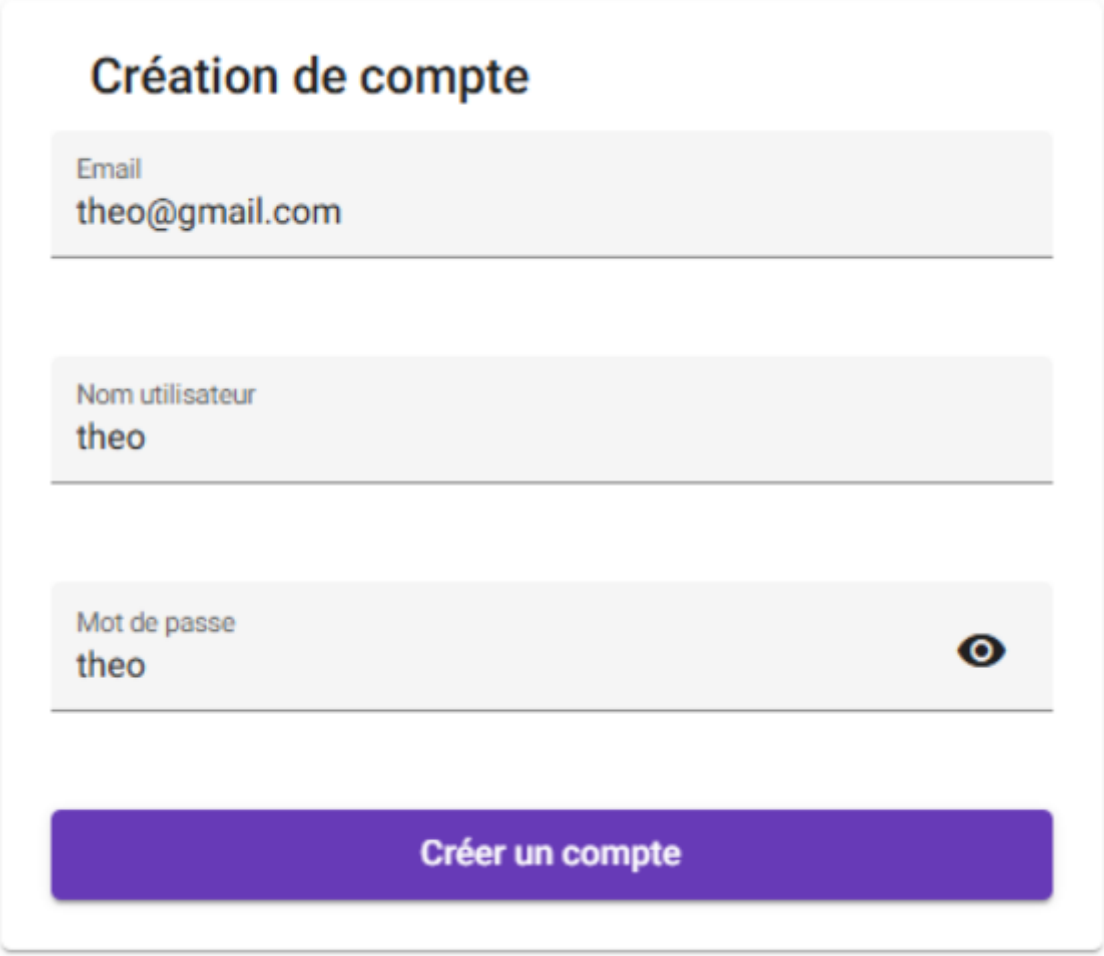

Figure 10 : Création d'un compte sur Truitteur

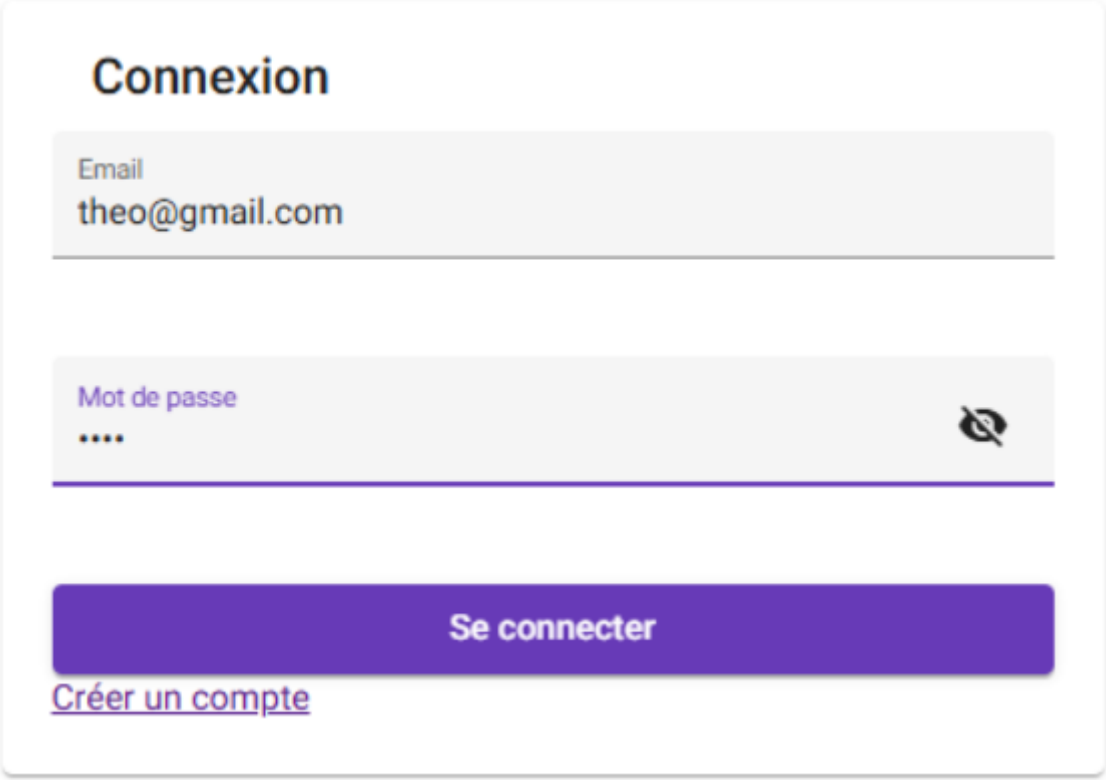

Figure 11 : Connexion à son compte Truitteur

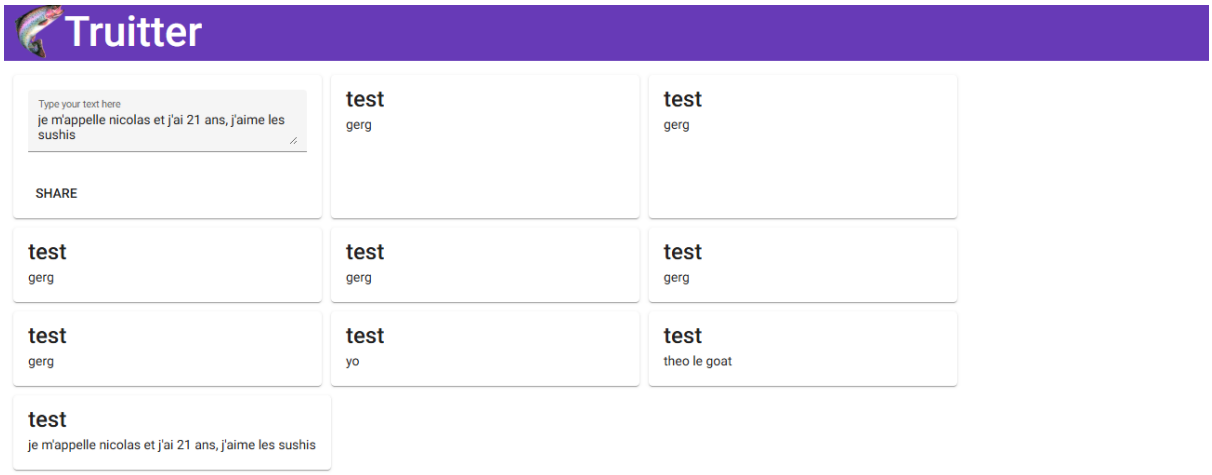

Figure 12 : Post d'un truitte sur Truitteur

IV. Conclusion

Pour conclure, ce projet nous a permis de mettre en pratique concrètement les concepts de DevSecOps. On est partis de zéro avec deux machines vides, et aujourd'hui nous avons une infrastructure fonctionnelle capable de déployer l'application Truitteur.

Nous avons réussi à remplir la mission principale : construire une chaîne CI/CD complète. C'était la partie la plus complexe et technique, notamment pour faire communiquer tous les outils (Jenkins, SonarQube, Nexus) dans des conteneurs Podman isolés. On a appris à gérer les problèmes de droits, de réseaux et à sécuriser les accès web avec Nginx.

Si nous devions continuer le projet, voici ce qu'il resterait à faire :

Avant le déploiement, porter notre attention sur les analyses de SonarQube et essayer de corriger les failles à la main. On pourrait aussi mettre un place un processus strict pour les gérer. Il s'agirait par exemple de faire en sorte de ne plus laisser passer le code si des failles critiques sont présentes. L'idée serait de trier les vulnérabilités détectées en utilisant le score CVSS et de prioriser celles qui font partie du Top 10 OWASP.

Après le déploiement, il s'agirait de vérifier que tout tourne et possiblement d'ajouter une phase de vérification de l'application en cours d'exécution (DAST).

Les difficultés qu'on s'attendrait à rencontrer seraient :

Avoir beaucoup de faux positifs et de bugs mineurs. Il aurait donc fallu faire une configuration pour ne filtrer que les failles les plus importantes (classées avec le score CVSS).

Le ralentissement de la production car les scans risquent d'être assez longs, et de plus en plus à chaque mise à jour.

Au global ce projet aura été une très bonne expérience pour l'ensemble du groupe qui nous a permis d'apprendre beaucoup de choses dans de nombreux domaines de la cyber.

Bibliographie

[1] G. Grevok, « Durcir la configuration SSH avec les bonnes pratiques ANSSI », *Ninja Linux*, 5 septembre 2025. [En ligne]. Disponible : https://ninjalinux.com/2025/09/05/durcir-la-configuration-ssh-avec-les-bonnes-pratiques-anssi/.

[2] « sshd_config(5) — OpenBSD manual page », *OpenBSD Manual Pages*. [En ligne]. Disponible : https://man.openbsd.org/sshd_config.

[3] C. Pécaud, « Tutoriel SSH sécurisé Ubuntu », *Tutoriels Pécaud Christopher*. [En ligne]. Disponible : https://tutoriels.pecaudchristopher.com/tutoriels_linux/espace_Ubuntu/espace_SystemeExploitation/Tutoriel_SSH_Securise_Ubuntu.php.

[4] D., « Durcissement de la configuration : comment sécuriser son serveur SSH », *IT-Connect*. [En ligne]. Disponible : https://www.it-connect.fr/durcissement-de-config-comment-securiser-son-serveur-ssh/.

[5] A. Jacob, « Sécuriser SSH sur un serveur Ubuntu 22.04 après son installation », *Blog Anthony Jacob*. [En ligne]. Disponible : https://blog.anthony-jacob.com/securiser-ssh-sur-un-serveur-ubuntu-22-04-apres-son-installation/.

[6] « Sécurité des mots de passe : ce que recommande l'ANSSI en 2025 », *Lockself Blog*, 23 avril 2025. [En ligne]. Disponible : https://www.lockself.com/blog/securite-mots-de-passe-anssi-2025.

[7] « Explication du fichier /var/log/auth.log sous Linux », *IT-Connect*. [En ligne]. Disponible : https://www.it-connect.fr/linux-explication-fichier-var-log-auth-log/.

[8] « Serveur dédié – Firewall et iptables », *OVHcloud Help*. [En ligne]. Disponible : https://help.ovhcloud.com/csm/fr-dedicated-servers-firewall-iptables?id=kb_article_view&sysparm_article=KB0043442.

[9] « How to install Java on Ubuntu », *OVHcloud Community Tutorials*. [En ligne]. Disponible : https://www.ovhcloud.com/fr/community/tutorials/how-to-install-java-ubuntu/.

[10] « How to install PostgreSQL on Ubuntu », *OVHcloud Community Tutorials*. [En ligne]. Disponible : https://www.ovhcloud.com/fr/community/tutorials/how-to-install-pg-ubuntu/.

[11] « J'ai créé une VM (futur serveur de prod), sécurisé la connexion SSH pour se connecter en clef privée uniquement et défini le pare-feu iptables le moins permissif possible. Je m'intéresse maintenant à l'installation de java 17 et postgresql 15. Donne-moi des indications sur comment les installer de la manière la plus propre en termes de sécurité informatique (security by design). Par exemple, définir quels utilisateurs ont le droit d'utiliser java ou comment faire pour sécuriser postgresql (moindre privilège). » à Google Gemini.

[12] T. Tran et V. Baranwal, « How To Configure Nginx as a Reverse Proxy on Ubuntu », *DigitalOcean Tutorials*, mise à jour le 6 octobre 2025. [En ligne]. Disponible : https://www.digitalocean.com/community/tutorials/how-to-configure-nginx-as-a-reverse-proxy-on-ubuntu-22-04.

[13] C. Maudoux, « Découverte & compréhension du serveur Web Nginx sous Debian », support de cours (TP), ESIEE Paris, cours « Gestion des identités et des accès », année scolaire 2025–2026.

[14] L. Perroton, « Prise en main d'OpenSSL : hachage et chiffrement », support de cours (TP), ESIEE Paris, année scolaire 2025–2026.

[15] OWASP Foundation, « HTTP Headers Cheat Sheet », *OWASP Cheat Sheet Series*. [En ligne]. Disponible : https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html.

[16] « Reverse proxy Nginx : guide de configuration », *Hostinger Tutoriels*. [En ligne]. Disponible : https://www.hostinger.com/fr/tutoriels/reverse-proxy-nginx#Conditions_prealables_a_la_configuration_dun_serveur_proxy_inverse_NGINX. [Consulté le 20 janvier 2026].

[17] « Podman vs Docker », *Red Hat – Topics: Containers*. [En ligne]. Disponible : https://www.redhat.com/en/topics/containers/podman-vs-docker.

[18] « Jenkins », *Jenkins Official Website*. [En ligne]. Disponible : https://www.jenkins.io/. [Consulté le 20 janvier 2026].

[19] « Jenkins Docker Image README », *GitHub – jenkinsci/docker*. [En ligne]. Disponible : https://github.com/jenkinsci/docker/blob/master/README.md.

[20] « Reverse proxy configuration with Nginx », *Jenkins Documentation – System Administration*. [En ligne]. Disponible : https://www.jenkins.io/doc/book/system-administration/reverse-proxy-configuration-nginx/.

[21] « Je souhaite lancer un conteneur Jenkins via Podman. Le conteneur doit être accessible derrière un reverse proxy Nginx avec le chemin '/jenkins'. Donne-moi la commande podman run complète incluant : Le mappage des ports 8080 et 50000. Un volume persistant vers le dossier hôte ~/jenkins_data. La configuration nécessaire pour définir le préfixe d'URL. L'attachement au réseau devops-net. » à Google Gemini.

[22] « Agis en tant qu'administrateur système expert Nginx. Rédige un fichier de configuration sites-available complet pour un serveur CI/CD. Il doit agir comme reverse-proxy pour trois services sur le même domaine via des chemins d'URL : Jenkins sur /jenkins (port 8080) : Doit supporter les WebSockets. SonarQube sur /sonar (port 9000). Nexus sur /nexus (port 8081) : Doit autoriser l'upload de fichiers jusqu'à 1Go. Le fichier doit inclure une redirection forcée HTTP vers HTTPS au début. » à Google Gemini.

[23] « SonarQube Docker Image », *Docker Hub – Official Images*. [En ligne]. Disponible : https://hub.docker.com/_/sonarqube.

[24] « SonarScanner », *SonarQube Documentation*. [En ligne]. Disponible : https://docs.sonarsource.com/sonarqube/latest/analyzing-source-code/scanners/sonarscanner/.

[25] « Elasticsearch permission denied », *Stack Overflow*. [En ligne]. Disponible : https://stackoverflow.com/questions/41956466/elasticsearch-permission-denied.

[26] « Installation et configuration de SonarQube », *YouTube*. [Vidéo en ligne]. Disponible : https://www.youtube.com/watch?v=7-P81EKq-r8.

[27] « J'essaie de lancer une analyse sonar-scanner sur mon serveur local, mais j'obtiens une erreur PKIX path building failed. Mon serveur SonarQube est derrière Nginx avec un certificat SSL auto-signé. Comment forcer le scanner Java à accepter ce certificat ou comment l'ajouter au truststore de la JVM ? » à Google Gemini.

[28] « Installation Methods », *Sonatype Help*. [En ligne]. Disponible : https://help.sonatype.com/en/installation-methods.html.

[29] « Nexus Repository OSS 3.87.1-01 for Linux x86_64 », *Sonatype Downloads*. [En ligne]. Disponible : https://download.sonatype.com/nexus/3/nexus-3.87.1-01-linux-x86_64.tar.gz.

[30] « Persistent data », *GitHub – sonatype/docker-nexus3*. [En ligne]. Disponible : https://github.com/sonatype/docker-nexus3#persistent-data.

[31] « J'essaie d'installer Nexus Repository Manager sur Ubuntu. La commande wget https://download.sonatype.com/nexus/3/latest-unix.tar.gz me renvoie une erreur 404 Not Found. Est-ce que ce lien existe toujours ? Comment trouver l'URL directe de la dernière version stable ? » à Google Gemini.

[32] « podman-run — Podman Manual Page », *Podman Documentation*. [En ligne]. Disponible : https://docs.podman.io/en/latest/markdown/podman-run.1.html.

[33] « Peux-tu me donner la syntaxe exacte du stage 'Build' pour compiler le projet avec Maven ? » à Google Gemini.

[34] « Eureka service always registers instances with host name even though I put prefer », *Stack Overflow*. [En ligne]. Disponible : https://stackoverflow.com/questions/62229903/eureka-service-always-registers-instances-with-host-name-even-though-i-put-pref.

[35] « Integrate Nexus with Jenkins | Automate Artifact Deployment via CICD », *YouTube*. [Vidéo en ligne]. Disponible : https://www.youtube.com/watch?v=8k8t_4jj7F0.

[36] « Pipeline Maven Integration », *Jenkins Plugins*. [En ligne]. Disponible : https://plugins.jenkins.io/pipeline-maven/.

[37] « Dans mon pipeline Jenkins, je veux intégrer l'analyse statique de code. Comment configurer l'étape SonarQube Analysis pour utiliser le wrapper withSonarQubeEnv() fourni par le plugin officiel SonarQube ? Je veux m'assurer que le scanner détecte automatiquement les variables de connexion (URL et Token) définies dans la configuration globale de Jenkins, plutôt que de les passer en dur dans une commande shell. » à Google Gemini.

[38] « Je cherche à optimiser mon pipeline CI/CD pour ne pas reconstruire inutilement une version déjà livrée. Écris-moi un script pour un Jenkinsfile qui interroge l'API REST de Nexus 3. Le but est de vérifier si un artefact existe déjà dans le dépôt maven-releases. Si l'artefact est trouvé (code HTTP 200), le pipeline doit ignorer les étapes de Build et Deploy. » à Google Gemini.

[39] M. Ozan Güven, « Running PostgreSQL with Podman », *Medium*. [En ligne]. Disponible : https://medium.com/@mehmetozanguven/running-postgresql-with-podman-4b71e31761b2.

[40] « Implement Quality Gates in Jenkins CICD Pipeline using SonarQube », *YouTube*. [Vidéo en ligne]. Disponible : https://www.youtube.com/watch?v=3t0U1ke6W00.

Annexes

Annexe A – Fichier /home/user/.ssh/Authorized_keys

Ce fichier contient la concaténation les clés publiques de chaque utilisateur pour que chacun puisse se connecter au compte de n'importe qui sur la VM. Ce fichier est le même sur le serveur de production et sur le serveur de compilation. Voir la partie 2.1.1.

AAAAB3NzaC1yc2EAAAADAQABAAABgQDSQUiS9do8BZfW31Ux05Q2NHWuGl9zlpXr+CBgDjBQe9k6Pr
iQJVe3jhu1Zly88nPNxOaj6kpdqGCRjl2Vvkjk/U7YXSFBf6zqJn2XzT9jXhXYuETl9kTqTzz3gj1/uk0GrKlB4d
M+tmUQk1lgnLpgX6T6D7dvSn5qE/lGEOx4Z4R26vr0k193TJrNk8VNHLgPjc8FikgTarTZVFzQc+fr2RTRcP
WrlLhIuTSBkXFhbGHaFaxTZgDgDlX4IGO5xEwZBeBxI3l9ApGfqM5HN1tEhVYH79ViewGJY3pHKCtscNDy
po0YduITFn7J7btK3TmzqrGfHSeXnSQDDcLoRiHiAc/yuM0ZqkMQNN9wrrGZR04hu0dhLxBsPvgpNhEeZ
03I0/ae67eFWY7/Sy5puWdZ7e7xrlnf8h/1jT1S6EVa4V0dSRoL8vvoOnabNm/1wYp+eFyrZ39bdc7yM5f
WuOQWqzE02xQoVmXVTqbrf0kZtlvzOnUCzRlFrV1/SXCBtYU= theo@ip-172-31-33-21

ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABgQDUiIjq4WzYt3jbdyLni5foj7TYaldsybZ9vltDGfdiBJjEoe8RRHICK
XQjkPt6ub93iHa9J4p8hEi3jyqagS6RqrR3ZpYubo5vABQ/VvueDLDtjkYAwTzpm6LzPDSLxPII36860RqlbV
49tCePRmshPKrCZ1yR2wEHSnx1NAXa0ljKJ1a2CDmJkuOr2Xsj/6C3uUuNk8T+Djyvj70MZEwxT819seZKf
7WTGqSyuGDu/j0ICI5BeFQEM/CYr9hG6QVh4NnLjPglsn3dWgpTTpsE4u1SkKZiY6YRTBLzUbIPVlzOjP32
qgeLO5IJvpk0LpeT6bjui0D7zbDZcrarVSQKOyDvjZQhXToeNpmmSb78xcoTPd4IIq9Y1Vyd2IY9giAM6lagL
OPHe0hHvISetmAe+q5WuFa2YrUT2LPOaH6h1v8BjLBoMsagStq+0z3n29SLLtRP1av+RdPgTn8slgI2wjaP
DmnjhG/UgS+ocUokVbCCIJWZFyjLxlv3iKE68mU= zelia@ip-172-31-33-21

ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABgQDeVdoZhnWrkMos7Le2OUikHUY5zF/kx/+Ng1KybuhCkafMCB
KQhX3aFOj+xMzstIi5mWuiXEeGet3/Ax0g5zAhT1YH9o+kJTLF6hquzRpAsv7gx2vcDWWIH56awJHYygA7
TiQsgNtIGDYG/7aZLPcW5alUAHrngnPA7YNjaRcQ43Wa+3OZU+Uhio1WB+w1c21EyYwiHNpCrFPn83uR
pKLbUEvqQ4dKKPAX1QnFO2QkcyN1qiD31HDe9lZKIIk2ArQWK/itISHwnyvyexQ7uWJHHQXBxyVLymmV
e+o4awLEWs1jolj6vInXV/nqvjDgCi/d8nOp9Dlaz13hI3I1cu25fnJsd2p9Nkw6ilwVdXtdoEVDt9jTm6E4qeL
a7gohXbiXvsq7x3qtjwt3+9nDJJGSJZCg/FJacz46pD8QaEYIp7YEm124I06Qr+QfTAIXkBvHneodejEcJqIyzZ
MRQYOOAKNlfL47/m4qKLMN1yjSbJ5WqtrktouOGvLH+E2YaSk= nicolas@ip-172-31-33-21

ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABgQClBDNIr7BV6iuyJor27W77xOc31Em5OGrQFEdhy40f5jPJnkW
7SOeBBn03k39YlakzoIQM0mdOVmPcTspywkkg82P/Ud1jbpBTqvuCGnYdPr/rZicGe1m3WhOdF44CmEa
Euu4pdL5qX/Y0wgU7xp85WEuGPBij6t0ochMLL8YAcR58LDvuPjpg++kFrNqSGqoZlGxthgqIGI3IVy9BLBp
cfJqx2WYQuj+y8XiWtOWPi/28adJ6qouWQ6VIRPz08sadO5GpCR8Cct/sFPwLQNGffDvOl2ArBELhr7Rua
15qIWCNfy40S+4mE2CjmplvgRGGhhhltFefXxmeYcExi9M18fQXkD29eAtMVd+9DKL7x13dSUtvKysNIKig
JMYDt30VDxKNhUnvOaB/sO92A49TVtW1GgGYzKDGiparhjPWuiTrFkYL8dC9ExOp4JUdEikZ7L2Np3HkN
H2koVhJAbfcL3XXwuY+QD5+/JXSZ9qCqlBTQ5mc7snnmQDezNOWP88= rayan@PC_de_Rayan

Annexe B – Modifications dans le fichier de configuration du service SSH

Toutes les lignes qui ont été ajoutées dans le fichier de configuration du service SSH /etc/ssh/sshd_config. Ces modifications sont les mêmes sur les serveurs de production et de compilation. Voir la partie 2.1.1.

```
PermitRootLogin no

PasswordAuthentication no

ClientAliveInterval 600

AllowGroups asystes

Port 52819
```

Annexe C – Fichier /var/log/auth.log

Un exemple de ce qui est contenu dans ce fichier. On y voit notamment les différentes tentatives de connexion à la machine virtuelle (en l'occurrence de production) depuis le service SSH. Voir la partie 2.1.1.

```
2026-01-25T14:17:02.552264+00:00 ip-172-31-33-21 CRON[1098]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)

2026-01-25T14:17:02.556018+00:00 ip-172-31-33-21 CRON[1098]: pam_unix(cron:session): session closed for user root

2026-01-25T14:24:07.780332+00:00 ip-172-31-33-21 polkitd[6271]: Loading rules from directory /etc/polkit-1/rules.d

2026-01-25T14:24:07.780399+00:00 ip-172-31-33-21 polkitd[6271]: Loading rules from directory /usr/share/polkit-1/rules.d

2026-01-25T14:35:01.568848+00:00 ip-172-31-33-21 CRON[7271]: pam_unix(cron:session): session closed for user root

2026-01-25T14:45:01.572749+00:00 ip-172-31-33-21 CRON[7451]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)

2026-01-25T14:45:01.574681+00:00 ip-172-31-33-21 CRON[7451]: pam_unix(cron:session): session closed for user root

2026-01-25T14:54:09.866515+00:00 ip-172-31-33-21 sshd[7598]: Server listening on 0.0.0.0 port 52819.

2026-01-25T14:54:09.866619+00:00 ip-172-31-33-21 sshd[7598]: Server listening on :: port 52819.
```

Annexe D – Commandes utilisées pour générer les biclés RSA

Voir la partie 2.1.1.

```
ssh-keygen -t rsa
cd .ssh/
cat id_rsa.pub > authorized_keys
chmod 600 authorized_keys
mv id_rsa.pub ~
cp -r id_rsa /tmp/
cd /tmp/
chmod 777 id_rsa
cd ~/.ssh/
rm id_rsa
```

Annexe E – Commandes utilisées pour la configuration du pare-feu Iptables

Les commandes tapées et leur détail. Ce sont les mêmes pour le serveur de production et le serveur de compilation. Voir la partie 2.1.2.

```
# 1. Réinitialisation des règles existantes
iptables -F
iptables -X

# ========================================
# RÈGLES D'ENTRÉE (INPUT)
# ========================================

# 2. Autoriser le trafic local (Loopback)
# Vital pour que Nginx (80) parle à SpringBoot (8080) et SpringBoot à Postgres (5432)
# sans exposer ces ports sur internet.
iptables -A INPUT -i lo -j ACCEPT
```

```
# 3. Maintenir les connexions déjà établies
# Permet au serveur de recevoir les réponses aux requêtes qu'il a lui-même initiées.
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# 4. Accès SSH (Port 52819)
# Note : Il est conseillé de restreindre cela à l'IP du serveur CI/CD et à celle des admins si fixes.
# Remplacez X.X.X.X par l'IP du serveur de compilation pour plus de sécurité.
# iptables -A INPUT -p tcp -s X.X.X.X --dport 52819 -j ACCEPT
iptables -A INPUT -p tcp --dport 52819 -j ACCEPT

# 5.Accès Web (Nginx Reverse-Proxy)
# On ouvre le 80 (HTTP) et le 443 (HTTPS) pour le public.
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Note : On N'OUVRE PAS les ports 8080 (Java) ou 5432 (Postgres) vers l'extérieur.
# Ils ne sont accessibles que via localhost (règle 2).

# ==========================================
# RÈGLES DE SORTIE (OUTPUT)
# ==========================================

# 6. Autoriser le trafic local sortant (Loopback)
iptables -A OUTPUT -o lo -j ACCEPT

# 7. Maintenir les connexions établies
iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# 8. Résolution DNS (UDP/TCP Port 53)
# Nécessaire pour que le serveur puisse résoudre les noms de domaine (apt, maven, etc.)
```

```
iptables -A OUTPUT -p udp --dport 53 -j ACCEPT

iptables -A OUTPUT -p tcp --dport 53 -j ACCEPT


# 9. Mises à jour et requêtes HTTP/HTTPS sortantes

# Nécessaire pour 'apt update', télécharger des dépendances ou curl.

iptables -A OUTPUT -p tcp --dport 80 -j ACCEPT

iptables -A OUTPUT -p tcp --dport 443 -j ACCEPT


# 10. Synchronisation temps (NTP - UDP 123)

# Important pour les logs et les certificats SSL.

iptables -A OUTPUT -p udp --dport 123 -j ACCEPT
```

Problème pour iptables persistent car pas autorisé input pour apt update donc pas possible d'installer paquet.

```
sudo iptables -I INPUT 1 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT


# 11. Politique par défaut : TOUT REFUSER (DROP)

iptables -P INPUT DROP

iptables -P FORWARD DROP

iptables -P OUTPUT DROP
```

Sauvegarde au redémarrage :

```
sudo apt-get install iptables-persistent

sudo netfilter-persistent save
```

Annexe F – Résultat de la commande « sudo iptables -L –line-numbers »

Voir la partie 2.1.2.

```
rayan@ip-172-31-33-21:~$ sudo iptables -L --line-numbers
Chain INPUT (policy DROP)
num  target     prot opt source               destination
1    ACCEPT     all  --  anywhere             anywhere             ctstate RELATED,ESTABLISHED
2    ACCEPT     all  --  anywhere             anywhere
3    ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:52819
4    ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:https
5    ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:http

Chain FORWARD (policy DROP)
num  target     prot opt source               destination

Chain OUTPUT (policy DROP)
num  target     prot opt source               destination
1    ACCEPT     all  --  anywhere             anywhere             ctstate RELATED,ESTABLISHED
2    ACCEPT     udp  --  anywhere             anywhere             udp dpt:domain
3    ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:domain
4    ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:http
5    ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:https
6    ACCEPT     all  --  anywhere             anywhere
7    ACCEPT     udp  --  anywhere             anywhere             udp dpt:ntp
```

Annexe G – Fichier de configuration d'Nginx dans le serveur de production

Il s'agit du fichier /etc/nginx/sites-available/truitteur. Voir partie 2.2.2.

```
server {
    # Redirection automatique HTTP vers HTTPS
    listen 80;
    listen [::]:80;
    #server_name;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    #server_name;

    # --- 1. CONFIGURATION SSL ---
    ssl_certificate /etc/nginx/ssl/nginx-selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key;
    ssl_protocols TLSv1.2 TLSv1.3; # On interdit les vieux protocoles (SSLv3, TLS1.0, 1.1)
    ssl_prefer_server_ciphers on;

    # --- 2. DURCISSEMENT ---
    server_tokens off; # Masque la version de Nginx

    # Limites pour éviter les DoS et Buffer Overflows
    client_body_buffer_size 10K;
    client_header_buffer_size 1k;
    client_max_body_size 8m; # Limite l'upload à 8MB
    large_client_header_buffers 2 1k;
```

```
# Timeouts agressifs pour couper les connexions mortes/lentes
client_body_timeout 12;
client_header_timeout 12;
keepalive_timeout 15;
send_timeout 10;

# --- 3. EN-TÊTES DE SÉCURITÉ (OWASP) ---
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always; # HSTS
add_header Referrer-Policy "no-referrer-when-downgrade" always;

# --- 4. RESTRICTIONS ---
# Interdire l'accès aux fichiers cachés (.git, .env, etc.)
location ~ /\. {
    deny all;
    access_log off;
    log_not_found off;
}

location / {
    proxy_pass http://localhost:8080;

    # Headers pour le proxy
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
```

```
        # Masquer les headers techniques de l'app Java si nécessaire
        proxy_hide_header X-Powered-By;
    }
}
```

Annexe H – Fichier de configuration de Sonar /opt/sonarqube/conf/sonar.properties

Il s'agit seulement de la partie « DATABASE » du fichier. Voir la partie 2.3.4.

```
#--------------------------------------------------------------------------------------------------
# DATABASE
#
# IMPORTANT:
# - The embedded H2 database is used by default. It is recommended for tests but not for
#   production use. Supported databases are Oracle, PostgreSQL and Microsoft SQLServer.
# - Changes to database connection URL (sonar.jdbc.url) can affect SonarSource licensed products.

# User credentials.
# Permissions to create tables, indices and triggers must be granted to JDBC user.
# The schema must be created first.
#sonar.jdbc.username=sonar
#sonar.jdbc.password=8_zF7n9Ua_Zrmf8pJ

#----- Embedded Database (default)
# H2 embedded database server listening port, defaults to 9092
#sonar.embeddedDatabase.port=9092



#----- Oracle 19c/21c
# The Oracle JDBC driver must be copied into the directory extensions/jdbc-driver/oracle/.
```

```
# Only the thin client is supported, and we recommend using the latest Oracle JDBC driver. See
# https://jira.sonarsource.com/browse/SONAR-9758 for more details.
# If you need to set the schema, please refer to http://jira.sonarsource.com/browse/SONAR-5000
#sonar.jdbc.url=jdbc:oracle:thin:@localhost:1521/XE


#----- PostgreSQL 11 or greater
# By default the schema named "public" is used. It can be overridden with the parameter "currentSchema".
#sonar.jdbc.url=jdbc:postgresql://localhost/sonarqube?currentSchema=my_schema


#----- Microsoft SQLServer 2014/2016/2017/2019/2022 and SQL Azure
# A database named sonar must exist and its collation must be case-sensitive (CS) and accent-sensitive (AS)
# Use the following connection string if you want to use integrated security with Microsoft Sql Server
# Do not set sonar.jdbc.username or sonar.jdbc.password property if you are using Integrated Security
# For Integrated Security to work, you have to download the Microsoft SQL JDBC Auth 11.2.2 package from
# https://github.com/microsoft/mssql-jdbc/releases/download/v11.2.2/mssql-jdbc_auth.zip
# and 'mssql-jdbc_auth-11.2.2.x64.dll' to your path.
#sonar.jdbc.url=jdbc:sqlserver://localhost;databaseName=sonar;integratedSecurity=true

# Use the following connection string if you want to use SQL Auth while connecting to MS Sql Server.
# Set the sonar.jdbc.username and sonar.jdbc.password appropriately.
#sonar.jdbc.url=jdbc:sqlserver://localhost;databaseName=sonar


#----- Connection pool settings
# The maximum number of active connections that can be allocated
```

```
# at the same time, or negative for no limit.
# The recommended value is 1.2 * max sizes of HTTP pools. For example if HTTP ports are
# enabled with default sizes (50, see property sonar.web.http.maxThreads)
# then sonar.jdbc.maxActive should be 1.2 * 50 = 60.
#sonar.jdbc.maxActive=60

# The minimum number of connections that can remain idle in the pool,
# without extra ones being created, or zero to create none.
#sonar.jdbc.minIdle=10

# The maximum number of milliseconds that the pool will wait (when there
# are no available connections) for a connection to be returned before
# throwing an exception, or <= 0 to wait indefinitely.
#sonar.jdbc.maxWait=8000
```

Annexe I – Fichier de configuration de Jenkins /var/lib/jenkins/config.xml

Voir la partie 2.3.3.

```
rayan@ip-172-31-34-68:~$ cat /var/lib/jenkins/config.xml
<?xml version='1.1' encoding='UTF-8'?>
<hudson>
  <disabledAdministrativeMonitors/>
  <version>2.528.3</version>
  <numExecutors>2</numExecutors>
  <mode>NORMAL</mode>
  <useSecurity>true</useSecurity>
  <authorizationStrategy class="hudson.security.FullControlOnceLoggedInAuthorizationStrategy">
    <denyAnonymousReadAccess>true</denyAnonymousReadAccess>
  </authorizationStrategy>
  <securityRealm class="hudson.security.HudsonPrivateSecurityRealm">
```

```
  <disableSignup>true</disableSignup>
  <enableCaptcha>false</enableCaptcha>
 </securityRealm>
 <disableRememberMe>false</disableRememberMe>
 <projectNamingStrategy
class="jenkins.model.ProjectNamingStrategy$DefaultProjectNamingStrategy"/>
 <workspaceDir>${JENKINS_HOME}/workspace/${ITEM_FULL_NAME}</workspaceDir>
 <buildsDir>${ITEM_ROOTDIR}/builds</buildsDir>
 <jdks/>
 <viewsTabBar class="hudson.views.DefaultViewsTabBar"/>
 <myViewsTabBar class="hudson.views.DefaultMyViewsTabBar"/>
 <clouds/>
 <scmCheckoutRetryCount>0</scmCheckoutRetryCount>
 <views>
  <hudson.model.AllView>
   <owner class="hudson" reference="../../.."/>
   <name>all</name>
   <filterExecutors>false</filterExecutors>
   <filterQueue>false</filterQueue>
   <properties class="hudson.model.View$PropertyList"/>
  </hudson.model.AllView>
 </views>
 <primaryView>all</primaryView>
 <slaveAgentPort>-1</slaveAgentPort>
 <label></label>
 <crumbIssuer class="hudson.security.csrf.DefaultCrumbIssuer">
  <excludeClientIPFromCrumb>false</excludeClientIPFromCrumb>
 </crumbIssuer>
 <nodeProperties/>
 <globalNodeProperties/>
```

```
  <nodeRenameMigrationNeeded>false</nodeRenameMigrationNeeded>
</hudson>
```

Annexe J - Fichier de configuration d'Nginx dans le serveur de compilation

Il s'agit du fichier /etc/nginx/sites-available/ci-cd. Voir partie 2.3.2.

```
upstream jenkins {
  keepalive 32;
  server 127.0.0.1:8080;
}

# Bloc 1 : Redirection forcée de HTTP vers HTTPS
server {
  listen 80;
  listen [::]:80;
  server_name _;
  return 301 https://$host$request_uri;
}

# Bloc 2 : Configuration HTTPS sécurisée
server {
  listen 443 ssl;
  listen [::]:443 ssl;
  server_name _;

  # Certificats SSL
  ssl_certificate /etc/nginx/ssl/nginx.crt;
  ssl_certificate_key /etc/nginx/ssl/nginx.key;
```

```
# Sécurité SSL (Protocoles forts uniquement)
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;

# Logs
access_log /var/log/nginx/jenkins.access.log;
error_log /var/log/nginx/jenkins.error.log;

# Masquer la version Nginx
server_tokens off;
ignore_invalid_headers off;

location /jenkins/foobar {
    proxy_pass        http://jenkins;
    proxy_redirect    default;
    proxy_http_version 1.1;

    # Headers obligatoires pour Jenkins + Websockets
    proxy_set_header  Connection       "upgrade";
    proxy_set_header  Upgrade          $http_upgrade;

    proxy_set_header  Host             $host;
    proxy_set_header  X-Real-IP        $remote_addr;
    proxy_set_header  X-Forwarded-For  $proxy_add_x_forwarded_for;
    proxy_set_header  X-Forwarded-Proto $scheme;

    # Augmentation upload et timeouts
    client_max_body_size      50m;
    client_body_buffer_size   128k;
    proxy_connect_timeout     90;
```

```
  proxy_send_timeout        90;
  proxy_read_timeout        90;
  proxy_request_buffering    off;
}

# --- Bloc SonarQube ---
location /sonar {
    proxy_pass http://127.0.0.1:9000/sonar;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

location /sonar/ {
    proxy_pass http://127.0.0.1:9000/sonar/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# --- Bloc Nexus ---
location /nexus {
    proxy_pass http://127.0.0.1:8081/nexus;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
```

```
        # Nexus a besoin de gros uploads (fichiers JAR)
        client_max_body_size 1G;
    }
}
```

Annexe K – Fichier de configuration de Nexus nexus.properties

Voir partie 2.3.5.

```
ubuntu@ip-172-31-34-68:~$ sudo cat ~/nexus_data/etc/nexus.properties
# Jetty section
# application-port=8081
# application-host=0.0.0.0
# nexus-args=${jetty.etc}/jetty.xml,${jetty.etc}/jetty-http.xml,${jetty.etc}/jetty-requestlog.xml
nexus-context-path=/nexus
```

Annexe L – Commandes pour la création des Trusstores

Voir partie 2.3.4.

```
keytool -importcert \
  -keystore /tmp/truitteur-truststore.jks \
  -storepass sonarpass \        //mot de passe du truststore
  -alias truitteur \
  -file /tmp/truitteur.crt \
  -noprompt
```

Annexe M – Création du conteneur Jenkins avec Podman

Voir la partie 2.3.3.

```
podman run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v ~/jenkins_data:/var/jenkins_home \
  --env JENKINS_OPTS="--prefix=/jenkins/foobar" \
  --restart=always \
  docker.io/jenkins/jenkins:lts-jdk17
```

-d : Detached (tourne en arrière-plan).

-p 8080:8080 : Mappe le port 8080 du conteneur sur le 8080 de la VM (donc Nginx pourra lui parler).

-v ... : Monte le dossier local dans le conteneur afin que tout ce que Jenkins sauvegarde aille dans ~/jenkins_data.

--env JENKINS_OPTS="..." : Configure le context path /jenkins/foobar.

Annexe N – Création du conteneur Nexus avec Podman

Voir la partie 2.3.5.

```
podman run -d
--name nexus
--network devops-net
-p 8081:8081
-v ~/nexus_data:/nexus-data
-e NEXUS_CONTEXT_PATH=/nexus
--restart=always docker.io/sonatype/nexus3
```

Annexe O – Création du conteneur PostgreSQL avec Podman

```
# Lancement du conteneur PostgreSQL pour SonarQube
podman run -d \
  --name sonar-db \
  --network devops-net \
  -e POSTGRES_USER=sonar \
  -e POSTGRES_PASSWORD=sonar_password_securise \
  -e POSTGRES_DB=sonar \
  -v ~/sonar_db:/var/lib/postgresql/data \
  docker.io/postgres:13
```

Annexe P – Création du conteneur SonarQube avec Podman

```
# Lancement du conteneur SonarQube
podman run -d \
  --name sonarqube \
  --network devops-net \
  -p 9000:9000 \
  -e SONAR_JDBC_URL=jdbc:postgresql://sonar-db:5432/sonar \
  -e SONAR_JDBC_USERNAME=sonar \
  -e SONAR_JDBC_PASSWORD=sonar_password_securise \
  -e SONAR_WEB_CONTEXT=/sonar \
  -v ~/sonar_data:/opt/sonarqube/data \
  --ulimit nofile=65536:65536 \
  docker.io/sonarqube:community
```

Annexe Q – Exemple de Pipeline Jenkins
Il s'agit ici du pipeline pour « Authentication-service »

```
pipeline {
  agent any

  tools {
    maven 'Maven 3'
    jdk 'Java 17'
  }

  environment {
    NEXUS_URL = 'http://nexus:8081/nexus/repository/maven-snapshots'
    SONAR_HOST_URL = 'http://sonarqube:9000'

    GROUP_ID = 'fr.esiee'
    ARTIFACT_ID = 'Gateway'
    VERSION = '0.0.1-SNAPSHOT'

    NEXUS_CREDS = credentials('nexus-auth')

    SONAR_TOKEN = credentials('sonar-token')

    GIT_CREDS_ID = 'git-auth'

    PROD_IP = '172.31.33.21'
    PROD_USER = 'app-deploy'
    PROD_CREDS = 'key_ssh_prod'
  }
```

```
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master',
                    credentialsId: GIT_CREDS_ID,
                    url: 'https://gitgud.totouiproject.ovh/cours_ESIEE/code/gateway.git'
            }
        }

        stage('Check Version in Nexus') {
            steps {
                script {
                    // 1. On récupère la version depuis le fichier pom.xml
                    def version = sh(script: 'mvn help:evaluate -Dexpression=project.version -q -DforceStdout',
returnStdout: true).trim()
                    echo "Version détectée : ${version}"
                    env.APP_VERSION = version

                    // 2. On demande à Nexus si ce fichier existe déjà
                    def urlToCheck = "${NEXUS_URL}/${GROUP_ID.replace('.',
'/')}/${ARTIFACT_ID}/${version}/${ARTIFACT_ID}-${version}.jar"

                    // On teste l'URL (Code 200 = existe, 404 = n'existe pas)
                    def status = sh(script: "curl -s -o /dev/null -w '%{http_code}' -u
${NEXUS_CREDS_USR}:${NEXUS_CREDS_PSW} ${urlToCheck}", returnStdout: true).trim()

                    if (status == '200' && !version.contains("SNAPSHOT")) {
                        echo "La RELEASE ${version} existe déjà. On ne touche à rien."
                        env.BUILD_NEEDED = 'false'
                    } else {
                        echo "Version inconnue OU Snapshot. On force la mise à jour."
```

```
                env.BUILD_NEEDED = 'true'
            }
        }
    }
}

    stage('Build & Test') {
        when { expression { return env.BUILD_NEEDED == 'true' } }
        steps {
            echo "Compilation en cours..."
            // On compile et on crée le JAR sans lancer les tests tout de suite
            sh 'mvn clean package -DskipTests'
        }
    }

stage('Debug Network') {
    steps {
        script {
            echo "--- TEST CONNEXION SONARQUBE ---"
            sh "curl -v http://sonarqube:9000/sonar"

            echo "--- TEST API VERSION ---"
            sh "curl -v http://sonarqube:9000/sonar/api/server/version"
        }
    }
}

    stage('SonarQube Analysis') {
        when { expression { return env.BUILD_NEEDED == 'true' } }
        steps {
```

```
            script {


                def scannerHome = tool name: 'sonar-scanner', type:
'hudson.plugins.sonar.SonarRunnerInstallation'


                withSonarQubeEnv('sonar-server') {
                  sh """
                  ${scannerHome}/bin/sonar-scanner \
                    -Dsonar.projectKey=${ARTIFACT_ID} \
                    -Dsonar.sources=src/main/java \
                    -Dsonar.java.binaries=target/classes \
                    -Dsonar.host.url=${SONAR_HOST_URL} \
                    -Dsonar.login=${SONAR_TOKEN}
                  """
                }
            }
        }
    }


    stage('Quality Gate') {
        when { expression { return env.BUILD_NEEDED == 'true' } }
        steps {
            script {
                timeout(time: 5, unit: 'MINUTES') {
                // Jenkins attend la réponse du Webhook de SonarQube
                waitForQualityGate abortPipeline: true
                }
            }
        }
    }
```

```
    stage('Publish to Nexus') {
      when { expression { return env.BUILD_NEEDED == 'true' } }
      steps {
        echo "Upload vers Nexus..."
        script {
          def targetUrl = "${NEXUS_URL}/${GROUP_ID.replace('.', '/')}/${ARTIFACT_ID}/${APP_VERSION}/${ARTIFACT_ID}-${APP_VERSION}.jar"

          // Upload via CURL
          sh """
            curl -v -u ${NEXUS_CREDS_USR}:${NEXUS_CREDS_PSW} --upload-file target/*.jar ${targetUrl}
          """
        }
      }
    }

    stage('Deploy to Prod') {
      steps {
        sshagent([PROD_CREDS]) {
          script {
            def remoteDir = "/home/${PROD_USER}/apps/gateway"

            // 1. Préparation du dossier
            echo "--- Préparation du serveur ---"
            sh "ssh -o StrictHostKeyChecking=no -p 52819 ${PROD_USER}@${PROD_IP} 'mkdir -p ${remoteDir}'"

            // 2. ENVOI DU FICHIER via SCP (C'est Jenkins qui pousse le fichier)
            echo "--- Transfert du JAR ---"
            // Attention : scp utilise -P (majuscule) pour le port, ssh utilise -p (minuscule)
```

```
                sh "scp -o StrictHostKeyChecking=no -P 52819 target/*.jar
${PROD_USER}@${PROD_IP}:${remoteDir}/app.jar"


                // 3. Redémarrage du conteneur
                echo "--- Redémarrage du service ---"
                sh """
                  ssh -o StrictHostKeyChecking=no -p 52819 ${PROD_USER}@${PROD_IP} "
                    podman stop gateway-service || true
                    podman rm gateway-service || true


                    podman run -d --name gateway-service \
                      --network esiee-network \
                      --hostname gateway-service \
                      -p 8080:8080 \
                      -v /home/app-deploy/apps/gateway/app.jar:/app/app.jar:Z \
                      -e EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://registry-
service:8761/eureka/ \
                      -e EUREKA_INSTANCE_PREFER_IP_ADDRESS=true \
                      -e
SPRING_CLOUD_GATEWAY_DISCOVERY_LOCATOR_LOWER_CASE_SERVICE_ID=true \
                      docker.io/library/eclipse-temurin:17-jre \
                      java -jar /app/app.jar
                  "
                """
              }
            }
        }
      }
    }
}
```